

AWS

IAM - Manages users, roles, groups, and policies to control who can access which AWS resources and what actions they can perform.

S3 - Scalable object storage service used to store and retrieve any amount of data such as files, backups, logs, and static website content.

EC2 - Virtual server (compute instance) in AWS that lets you run applications on scalable infrastructure without owning physical hardware.

Elastic BeanStalk - PaaS service that automatically handles deployment, capacity provisioning, load balancing, and scaling of your application.

AWS Lambda - Serverless compute service that runs your code in response to events without requiring you to provision or manage servers.

AWS Dynamo DB - Fully managed NoSQL database service offering single-digit millisecond latency with automatic scaling and high availability.

AWS Secret Manager - Securely stores, rotates, and manages sensitive information like database passwords, API keys, and credentials.

AWS Certificate Manager - Manages provisioning, renewal, and deployment of SSL/TLS certificates for secure HTTPS communication.

AWS RDS — Managed relational database service supporting engines like MySQL, PostgreSQL, MariaDB, Oracle, and SQL Server with automated backups and patching.

SQS — Fully managed message queue service used to decouple and scale microservices through asynchronous communication between components.

SNS — Pub/Sub notification service that sends messages to multiple subscribers simultaneously via email, SMS, Lambda, or SQS.

EKS — AWS managed Kubernetes service that simplifies deploying, managing, and scaling containerized applications using Kubernetes.

ECS — AWS native container orchestration service for running, stopping, and managing Docker containers on a cluster.

Aurora DB — High-performance, fully managed relational database compatible with MySQL and PostgreSQL, offering up to 5x MySQL speed.

AWS MSK — Fully managed Apache Kafka service that handles cluster provisioning, configuration, and maintenance for real-time streaming applications.

Route 53 — Highly available and scalable AWS DNS service used for domain registration, routing, and health checking.

CloudFront — Content Delivery Network (CDN) service that delivers data, videos, and applications globally with low latency using edge locations.

Fargate — Serverless compute engine for running containers without managing the underlying EC2 server infrastructure.

ELB (Elastic Load Balancer) — Automatically distributes incoming application traffic across multiple targets like EC2 instances for high availability and fault tolerance.

ElastiCache — Fully managed in-memory caching service supporting Redis and Memcached to improve application read performance.

CloudWatch — Monitoring and observability service used to collect metrics, logs, and alarms for AWS resources and applications.

CloudTrail — Governance and auditing service that records and logs all API calls and account activity across your AWS infrastructure.

VPC (Virtual Private Cloud) — A logically isolated virtual network in AWS where you launch resources with full control over IP ranges, subnets, route tables, and gateways.

Subnet — A range of IP addresses within a VPC; public subnets have internet access via an Internet Gateway, private subnets do not.

Internet Gateway (IGW) — A horizontally scaled, redundant VPC component that enables communication between resources in a public subnet and the internet.

NAT Gateway — Allows resources in a private subnet to initiate outbound internet connections (e.g., software updates) without being directly reachable from the internet.

Security Group — A stateful virtual firewall that controls inbound and outbound traffic at the instance level using allow-only rules.

Network ACL (NACL) — A stateless firewall that controls inbound and outbound traffic at the subnet level; supports both allow and deny rules evaluated in order.

Route Table — A set of rules that determines where network traffic from a subnet or gateway is directed within a VPC.

VPC Peering — A networking connection between two VPCs allowing private traffic routing between them without traversing the public internet.

AWS Transit Gateway — A network hub that connects multiple VPCs, on-premises networks, and AWS accounts through a single central gateway.

Direct Connect — A dedicated, private network connection between an on-premises data center and AWS, bypassing the public internet for consistent bandwidth and lower latency.

VPN (Site-to-Site) — An encrypted IPSec tunnel between an on-premises network and a VPC, providing secure connectivity over the public internet.

Availability Zone (AZ) — An isolated data center (or group of data centers) within an AWS Region with independent power, cooling, and networking.

AWS Region — A geographic area consisting of multiple Availability Zones, allowing you to deploy applications close to users and meet data residency requirements.

Edge Location — A CloudFront data center closer to end users used to cache and deliver content with low latency.

Auto Scaling Group (ASG) — Automatically adjusts the number of EC2 instances up or down based on demand, health checks, or scheduled policies.

Launch Template — A versioned EC2 configuration blueprint specifying instance type, AMI, key pair, security groups, and user data for Auto Scaling or direct launch.

AMI (Amazon Machine Image) — A pre-configured snapshot of an EC2 instance including OS, application server, and application data used to launch new instances.

EC2 Instance Types — Categories of EC2 instances optimized for different workloads: General Purpose (t, m), Compute Optimized (c), Memory Optimized (r), Storage Optimized (i), GPU (p, g).

Reserved Instances — A billing commitment (1 or 3 years) for a specific EC2 instance type offering up to 72% discount over On-Demand pricing.

Spot Instances — Unused EC2 capacity available at up to 90% discount; can be interrupted by AWS with a 2-minute warning when capacity is needed.

On-Demand Instances — EC2 instances billed per second with no upfront commitment; most flexible but most expensive pricing model.

Savings Plans — A flexible pricing model offering lower rates than On-Demand in exchange for a consistent usage commitment (\$/hour) over 1 or 3 years.

EBS (Elastic Block Store) — Persistent block-level storage volumes attached to EC2 instances, surviving instance restarts and supporting snapshots.

EBS Snapshot — A point-in-time incremental backup of an EBS volume stored in S3, used for disaster recovery or creating new volumes.

EFS (Elastic File System) — A fully managed, scalable NFS file system that can be mounted concurrently by multiple EC2 instances across Availability Zones.

S3 Bucket — A top-level container in S3 where objects (files) are stored; globally unique name required, region-specific, with configurable access policies.

S3 Object — A file stored in S3 consisting of data, a unique key (path), metadata, and optionally a version ID; maximum size is 5TB.

S3 Storage Classes — Different tiers offering cost/access trade-offs: Standard, Intelligent-Tiering, Standard-IA, One Zone-IA, Glacier, and Glacier Deep Archive.

S3 Glacier — A low-cost S3 storage class for archiving data that is rarely accessed; retrieval times range from minutes to hours depending on the tier.

S3 Lifecycle Policy — Automated rules that transition objects between storage classes or expire/delete them after a defined number of days.

S3 Versioning — A bucket-level feature that preserves, retrieves, and restores every version of every object, protecting against accidental deletion or overwrite.

S3 Replication (CRR/SRR) — Automatically copies objects from one S3 bucket to another in the same region (SRR) or a different region (CRR) for redundancy or compliance.

S3 Pre-Signed URL — A time-limited URL that grants temporary access to a private S3 object without requiring AWS credentials, used for secure file sharing.

S3 Transfer Acceleration — Speeds up long-distance S3 uploads by routing data through CloudFront edge locations to AWS over an optimized network path.

IAM Role — An identity with a set of permissions not tied to a specific user, assumed temporarily by AWS services, EC2 instances, or external identities.

IAM Policy — A JSON document attached to users, groups, or roles that explicitly allows or denies specific AWS API actions on specific resources.

IAM Group — A collection of IAM users that all share the same set of attached permissions policies for easier management.

IAM User — An identity representing a person or application with long-term credentials (access key and secret) for programmatic or console access.

STS (Security Token Service) — Generates temporary, limited-privilege AWS credentials for IAM roles, federated users, or cross-account access.

Resource-Based Policy — An IAM policy attached directly to an AWS resource (e.g., S3 bucket policy, Lambda resource policy) defining who can access it.

AWS Organizations — A service for centrally managing multiple AWS accounts, enabling consolidated billing, SCPs, and organizational unit hierarchies.

SCP (Service Control Policy) — An organization-level policy that sets the maximum permissions allowed for accounts within an AWS Organizations OU.

AWS Config — A service that continuously monitors and records AWS resource configurations and evaluates them against desired compliance rules.

AWS Systems Manager (SSM) — A management service for viewing operational data, automating tasks, patching, and accessing EC2 instances without SSH using Session Manager.

Parameter Store — An SSM feature for storing configuration data and secrets as plain text or encrypted values, accessible by applications and Lambda functions.

AWS KMS (Key Management Service) — A managed service for creating, managing, and using cryptographic keys to encrypt AWS resources and data at rest.

CMK (Customer Master Key) — A KMS key used to encrypt and decrypt data; can be AWS-managed or customer-managed with full rotation and policy control.

AWS WAF (Web Application Firewall) — Protects web applications from common exploits like SQL injection and XSS by filtering HTTP requests based on configurable rules.

AWS Shield — A managed DDoS protection service; Shield Standard is free for all, Shield Advanced provides enhanced protection and 24/7 DDoS response team access.

AWS GuardDuty — A threat detection service that continuously monitors AWS accounts for malicious activity using ML, anomaly detection, and threat intelligence.

AWS Inspector — An automated security assessment service that scans EC2 instances and container images for software vulnerabilities and unintended network exposure.

AWS Macie — A data security service that uses ML to automatically discover, classify, and protect sensitive data (PII, credentials) stored in S3.

Kinesis Data Streams — A real-time data streaming service for collecting and processing large streams of data records from multiple producers with low latency.

Kinesis Firehose — A fully managed service that loads streaming data directly into destinations like S3, Redshift, Elasticsearch, or Splunk without managing consumers.

Kinesis Data Analytics — Runs SQL or Apache Flink queries on real-time streaming data from Kinesis Streams or Firehose for real-time analytics.

AWS Glue — A fully managed serverless ETL (Extract, Transform, Load) service for discovering, cataloging, and transforming data for analytics.

AWS Athena — A serverless, interactive query service that analyzes data stored directly in S3 using standard SQL without loading it into a database.

AWS Redshift — A fully managed, petabyte-scale cloud data warehouse optimized for OLAP queries and business intelligence workloads.

AWS EMR (Elastic MapReduce) — A managed big data platform for running Apache Spark, Hadoop, Hive, and other frameworks on dynamically scalable EC2 clusters.

AWS Step Functions — A serverless workflow orchestration service that coordinates multiple AWS services into visual state machine workflows with retries and error handling.

EventBridge — A serverless event bus that routes events from AWS services, SaaS apps, and custom applications to targets like Lambda, SQS, or Step Functions.

AppSync — A fully managed GraphQL API service that connects apps to data sources like DynamoDB, Lambda, and HTTP APIs with real-time and offline capabilities.

API Gateway — A fully managed service for creating, publishing, securing, and monitoring REST, HTTP, and WebSocket APIs at any scale.

AWS Cognito — A user identity and authentication service providing sign-up, sign-in, MFA, social federation, and JWT token issuance for web and mobile apps.

Cognito User Pool — A managed user directory in Cognito that handles registration, login, password reset, and returns JWT tokens upon successful authentication.

Cognito Identity Pool — Provides temporary AWS credentials to authenticated or guest users, enabling direct access to AWS services like S3 or DynamoDB from the client.

AWS SAM (Serverless Application Model) — An open-source framework extending CloudFormation for defining, deploying, and testing serverless applications locally and in AWS.

CloudFormation — An Infrastructure as Code service that provisions and manages AWS resources from declarative JSON or YAML templates.

CDK (Cloud Development Kit) — An open-source framework for defining AWS cloud infrastructure using familiar programming languages like TypeScript, Python, and Java.

Terraform — A popular open-source, cloud-agnostic IaC tool for provisioning and managing AWS and multi-cloud infrastructure using HCL configuration files.

AWS CodePipeline — A fully managed CI/CD service that automates the build, test, and deploy phases of release processes triggered by code changes.

AWS CodeBuild — A fully managed build service that compiles code, runs tests, and produces deployable artifacts without managing build servers.

AWS CodeDeploy — A deployment service that automates application deployments to EC2, ECS, Lambda, or on-premises servers with rollback support.

AWS CodeCommit — A fully managed, private Git repository hosting service integrated with AWS IAM for access control (now limited to existing users).

ECR (Elastic Container Registry) — A fully managed Docker container image registry for storing, versioning, and deploying container images securely within AWS.

AWS Backup — A centralized, fully managed backup service that automates and manages backups across AWS services like EBS, RDS, DynamoDB, and EFS.

AWS Cost Explorer — A tool for visualizing, analyzing, and forecasting AWS spending and usage patterns to identify cost optimization opportunities.

AWS Budgets — Allows setting custom cost and usage budget thresholds with alerts via SNS when actual or forecasted spend exceeds defined limits.

AWS Trusted Advisor — A real-time guidance tool that analyzes your AWS environment and recommends improvements in cost optimization, security, performance, and fault tolerance.

AWS Well-Architected Framework — A set of best practices across five pillars (Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization) for building robust cloud systems.

Shared Responsibility Model — AWS's security model where AWS is responsible for security OF the cloud (infrastructure) and customers are responsible for security IN the cloud (data, IAM, apps).

AWS Global Accelerator — A networking service that routes user traffic through AWS's global private network to the nearest healthy endpoint, improving availability and latency.

AWS PrivateLink — Enables private connectivity between VPCs and AWS services or third-party services without exposing traffic to the public internet.

S3 Access Points — Named network endpoints attached to S3 buckets that simplify managing access for large shared datasets with per-application policies.

DynamoDB Streams — A time-ordered log of item-level changes in a DynamoDB table, enabling event-driven processing with Lambda triggers.

DynamoDB Global Tables — A fully managed, multi-region, multi-active replication feature that keeps DynamoDB tables synchronized across AWS Regions.

DynamoDB DAX — An in-memory cache for DynamoDB that delivers microsecond read latency for eventually consistent reads without application code changes.

RDS Multi-AZ — A high-availability feature that automatically maintains a synchronous standby replica in a different AZ and fails over automatically on instance failure.

RDS Read Replica — An asynchronous read-only copy of an RDS instance used to offload read traffic, improve performance, and support cross-region disaster recovery.

RDS Proxy — A fully managed database proxy for RDS and Aurora that pools and shares connections, improving scalability and reducing failover time for serverless apps.

ElastiCache Redis — An in-memory data store used for caching, session management, leaderboards, pub/sub messaging, and real-time analytics with persistence support.

ElastiCache Memcached — A simple, high-performance, multi-threaded in-memory caching engine for straightforward object caching without persistence or advanced features.

SQS Standard Queue — A fully managed queue with at-least-once delivery, best-effort ordering, and virtually unlimited throughput for high-scale decoupled applications.

SQS FIFO Queue — A queue that guarantees exactly-once processing and strict first-in-first-out message ordering, with a throughput limit of 3,000 messages/second with batching.

SQS Dead Letter Queue (DLQ) — A queue that receives messages that could not be processed successfully after a maximum number of receive attempts, for debugging and reprocessing.

SQS Visibility Timeout — The period during which a received message is hidden from other consumers, allowing the processing consumer time to delete it before it reappears.

SNS Topic — A logical access point and communication channel in SNS where publishers send messages and subscribers (Lambda, SQS, email, HTTP) receive them.

SNS Fan-Out Pattern — A messaging pattern where a single SNS topic delivers a message to multiple SQS queues simultaneously, enabling parallel processing by different services.

Lambda Trigger — An event source (S3, DynamoDB, SQS, API Gateway, EventBridge, etc.) that automatically invokes a Lambda function when a specific event occurs.

Lambda Layers — A mechanism to package and share common libraries, runtimes, or configurations across multiple Lambda functions without including them in each deployment package.

Lambda Concurrency — The number of function instances processing requests simultaneously; can be reserved or provisioned to control scaling and avoid cold starts.

Lambda Cold Start — The initialization delay when a Lambda function is invoked for the first time or after being idle, caused by provisioning and bootstrapping the execution environment.

Lambda Provisioned Concurrency — Pre-initializes a specified number of Lambda execution environments to eliminate cold starts for latency-sensitive applications.

Lambda@Edge — Runs Lambda functions at CloudFront edge locations to process HTTP requests and responses close to users with very low latency.

Kubernetes

Pod — Smallest deployable unit in Kubernetes that encapsulates one or more containers sharing the same network and storage.

Node — A worker machine (VM or physical server) in a Kubernetes cluster where pods are scheduled and run.

Cluster — A set of master and worker nodes that together run and manage containerized applications.

Deployment — Manages pod replicas and enables rolling updates and rollback.

ReplicaSet — Ensures a specified number of identical pod replicas are always running at any given time.

Service — Provides a stable network endpoint and load balancing to expose pods internally or externally.

ClusterIP — Internal service accessible only inside the cluster.

NodePort — Exposes service externally using a port on each node.

LoadBalancer — Exposes service externally using cloud provider load balancer.

Ingress — Manages external HTTP/HTTPS routing to services using rules.

ConfigMap — Stores non-sensitive configuration data as key-value pairs.

Secret — Securely stores sensitive data like passwords and tokens.

Namespace — Logical isolation used to organize resources inside a cluster.

kubectl — CLI tool used to interact with Kubernetes clusters.

etcd — Distributed key-value store used to store Kubernetes cluster state.

Kubelet — Agent running on each node that manages pods and containers.

Kube-Proxy — Handles networking and traffic routing inside the cluster.

Control Plane (Master Node) — The brain of the Kubernetes cluster that manages scheduling, health monitoring, and overall cluster decisions.

Scheduler — Control plane component that decides which worker node a newly created pod should run on based on resource availability.

API Server — Entry point for all Kubernetes API requests.

Controller Manager — Runs built-in controllers (like node, replication, endpoint controllers) to maintain the desired cluster state.

StatefulSet — Manages stateful applications with stable identity and storage.

DaemonSet — Ensures exactly one pod runs on every node in the cluster, commonly used for log collectors and monitoring agents.

Job — Runs a task until completion successfully.

CronJob — Runs Jobs on a scheduled basis using cron syntax, useful for periodic tasks like backups or report generation.

Persistent Volume (PV) — A piece of storage in the cluster provisioned by an admin or dynamically, independent of any pod lifecycle.

Persistent Volume Claim (PVC) — A request made by a pod for a specific amount and type of storage from available Persistent Volumes.

StorageClass — Defines how storage is dynamically provisioned including the provider, type, and reclaim policy.

Helm — Kubernetes package manager that uses charts to define, install, and upgrade complex Kubernetes applications.

Horizontal Pod Autoscaler (HPA) — Automatically scales pods based on CPU/memory usage.

Taint — Restricts pods from scheduling on certain nodes unless tolerated.

Toleration — Allows pods to run on tainted nodes.

Label — Key-value metadata attached to Kubernetes objects to identify, group, and select resources.

Selector — Used to filter and match Kubernetes resources based on their labels for grouping or targeting.

Sidecar Container — A helper container that runs alongside the main application container in the same pod to extend functionality.

Init Container — A container that runs and must complete successfully before the main application container starts.

Liveness Probe — Periodically checks if a container is still running and restarts it if the check fails.

Readiness Probe — Checks if a pod is ready to serve traffic and removes it from the service endpoint if the check fails.

Resource Limits — Maximum CPU and memory a container is allowed to use, enforced by the Kubernetes scheduler.

Resource Requests — Minimum CPU and memory guaranteed to a container, used by the scheduler to place pods on nodes.

RBAC — Role-Based Access Control system that regulates which users or service accounts can perform which actions on Kubernetes resources.

Network Policy — Kubernetes resource that defines rules for controlling inbound and outbound pod-to-pod network traffic.

Minikube — Local single-node Kubernetes cluster tool for development and testing on a personal machine.

EKS — AWS managed Kubernetes service.

Kafka

Kafka — Distributed event streaming platform used for real-time data pipelines and messaging.

Producer — Application that sends messages to Kafka topics.

Consumer — Application that reads messages from Kafka topics.

Topic — Logical channel or category to which producers publish messages and from which consumers read.

Partition — A topic is divided into partitions for parallel processing, scalability, and ordered message storage.

Broker — A Kafka server that stores messages and serves producer and consumer requests within the cluster.

Cluster — Group of Kafka brokers working together.

Offset — A unique sequential ID assigned to each message within a partition to track its position.

Consumer Group — Group of consumers sharing message consumption workload.

Replication — The process of copying partition data across multiple brokers to ensure fault tolerance and data durability.

Leader Partition — The primary replica of a partition responsible for all reads and writes; other replicas follow it.

Follower Partition — A replica that continuously syncs data from the leader partition and takes over if the leader fails.

ZooKeeper — External coordination service previously used by Kafka for managing broker metadata, leader election, and configuration.

KRaft — Kafka's built-in consensus protocol that replaces ZooKeeper for managing cluster metadata since Kafka 2.8+.

Retention Period — Time Kafka keeps messages before deletion.

ACK — Acknowledgment setting on a producer that controls how many broker replicas must confirm receipt before the write is considered successful.

ISR (In-Sync Replica) — Replicas fully synced with leader partition.

Lag — The gap between the latest offset in a partition and the last offset processed by a consumer; indicates how far behind a consumer is.

Rebalancing — The process of redistributing partition assignments among consumers in a group when members join or leave.

Serializer — Converts objects into bytes before sending to Kafka.

Deserializer — Converts Kafka bytes back into objects for consumers.

Idempotent Producer — A producer configuration that ensures each message is written exactly once to a partition, preventing duplicates on retry.

Exactly Once Semantics — Guarantees that a message is produced, delivered, and consumed exactly once with no duplication or data loss.

At Least Once — Messages may duplicate but are never lost.

At Most Once — Messages may be lost but never duplicated.

Dead Letter Topic (DLQ) — A special Kafka topic used to capture messages that failed processing so they can be analyzed or reprocessed.

Kafka Connect — Tool for integrating Kafka with external systems/databases.

Kafka Streams — Library for stream processing directly within Kafka applications.

Schema Registry — Centralized service that stores and enforces Avro/JSON/Protobuf schemas for Kafka message serialization consistency.

Compaction — Log compaction strategy that retains only the latest message per key in a partition, removing outdated duplicates.

Batching — Producer sending multiple messages together for better performance.

Throughput — Number of messages processed per second.

Consumer Polling — Consumers repeatedly call the poll() method to fetch batches of messages from Kafka brokers.

Commit Offset — The action of saving the offset of the last successfully processed message so the consumer can resume correctly after a restart.

Synchronous Communication — Sender waits for immediate response.

Asynchronous Communication — Sender continues without waiting for response.

Event Streaming — Continuous flow and processing of real-time events/data.

Back Pressure — A condition where the consumer cannot process messages as fast as the producer sends them, causing queue build-up.

Ordering Guarantee — Kafka guarantees message order only within a single partition, not across partitions of a topic.

Horizontal Scaling — Adding more partitions or brokers to a Kafka cluster to increase throughput and capacity.

Fault Tolerance — Kafka's ability to continue operating correctly even when one or more brokers fail, due to replication.

Microservices

Microservices — Architecture where applications are split into small independently deployable services.

Monolith — Single large application where all modules are tightly coupled.

Service Discovery — Service Discovery — A mechanism that allows services to dynamically find and communicate with each other without hardcoded addresses.

Eureka Server — Netflix OSS service registry where microservices register themselves and discover other services at runtime.

API Gateway — Single entry point for all client requests, handling routing, authentication, rate limiting, and load balancing.

Load Balancing — Distributes incoming requests evenly across multiple service instances to avoid overloading any single instance.

Feign Client — Declarative REST client for service-to-service communication.

REST API — Communication style using HTTP methods like GET, POST, PUT, DELETE.

Synchronous Communication — A microservice directly calls another and waits for a response before proceeding.

Asynchronous Communication — A microservice sends an event or message to a queue/broker without waiting for the consumer to process it.

Circuit Breaker — A resilience pattern that stops forwarding requests to a failing service and returns a fallback to prevent cascading failures.

Retry Mechanism — Automatically re-attempts a failed service call a configured number of times before giving up.

Fallback — A predefined alternative response or action executed when a service call fails or times out.

Resilience4j — A lightweight Java fault-tolerance library providing circuit breaker, retry, rate limiter, and bulkhead patterns.

Distributed Tracing — Technique for tracking and visualizing the full journey of a request as it passes through multiple microservices.

Sleuth — Spring Cloud library that automatically adds trace IDs and span IDs to logs for distributed request tracing.

Zipkin — Open-source distributed tracing system that collects, stores, and visualizes timing data across microservices.

Centralized Logging — Aggregating logs from all microservices into a single platform for unified monitoring and debugging.

ELK Stack — Combination of Elasticsearch (search), Logstash (ingestion), and Kibana (visualization) for centralized log management.

Config Server — Centralized service that provides externalized configuration for all microservices from a single source (e.g., Git).

Spring Cloud Config — Externalized centralized configuration service.

Docker — Containerization platform that packages an application and its dependencies into a portable image that runs consistently anywhere.

Containerization — Bundling application code, libraries, and runtime into a container for consistent, environment-independent deployment.

Kubernetes — Open-source container orchestration platform for automating deployment, scaling, and management of containerized applications.

Horizontal Scaling — Increasing the number of service instances to handle more traffic instead of upgrading server hardware.

Vertical Scaling — Increasing the CPU or memory capacity of an existing server to improve its performance.

Database Per Service — Design principle where each microservice manages its own dedicated database to maintain loose coupling.

Distributed Transaction — A transaction that spans multiple microservices or databases and must remain consistent across all of them.

Saga Pattern — Manages distributed transactions by breaking them into a series of local transactions with compensating actions on failure.

Event-Driven Architecture — Design pattern where services communicate by producing and consuming events through a message broker.

Strangler Fig Pattern — Gradually replaces a legacy monolith by routing new functionality to microservices while keeping the old system running in parallel until fully replaced.

Outbox Pattern — Ensures atomicity between a database write and a message publish by writing the event to an outbox table in the same transaction, then publishing it asynchronously.

Sidecar Pattern — Deploys a helper process (e.g., logging agent, proxy) alongside the main service container in the same pod to offload cross-cutting concerns.

Aggregator Pattern — A service that calls multiple downstream microservices, collects their responses, and returns a single combined result to the client.

Chained Microservice Pattern — A request passes through a sequence of services in a chain where each service processes and passes output to the next.

Event Sourcing — Persists the state of an entity as an ordered sequence of immutable domain events rather than the current state, enabling full audit trail and replay.

Two-Phase Commit (2PC) — A distributed transaction protocol where a coordinator asks all participants to prepare, then commits only if all agree; avoided in microservices due to tight coupling and blocking.

Compensating Transaction — An operation that semantically undoes a previously completed step in a saga when a later step fails, restoring business consistency without true rollback.

Service Mesh Pattern — Offloads inter-service communication concerns (load balancing, retries, mTLS, tracing) to a dedicated infrastructure layer like Istio, freeing services from that logic.

Backends for Frontends (BFF) — Creates a separate API gateway/backend tailored for each type of client (mobile, web, third-party), avoiding a one-size-fits-all API.

Throttling Pattern — Limits the rate at which a service processes requests from a client to protect against overload and ensure fair resource usage.

Rate Limiting — Enforces a maximum number of requests a client can make in a given time window, commonly implemented at the API Gateway.

Health Check Pattern — Each microservice exposes a /health endpoint that reports its liveness and readiness status, used by load balancers and orchestrators to route traffic.

Leader Election Pattern — When multiple instances of a service run concurrently, one is elected as leader to perform a singleton task (e.g., scheduled job), preventing duplication.

Choreography (Saga) — Services react to domain events published by other services without a central coordinator; each service knows what to do when it receives a specific event.

Orchestration (Saga) — A central saga orchestrator explicitly tells each service what to do and handles compensation logic when a step fails.

API Composition Pattern — A query is handled by calling multiple services and joining the results in memory at the API layer, used when data spans multiple service databases.

Gateway Aggregation — The API Gateway aggregates multiple fine-grained service calls into a single response, reducing client round trips.

Gateway Offloading — Cross-cutting concerns like authentication, SSL termination, logging, and rate limiting are offloaded to the API Gateway instead of each service.

Client-Side Load Balancing — The client fetches the service instance list from the registry and decides which instance to call (e.g., Ribbon, Spring Cloud LoadBalancer).

Server-Side Load Balancing — A load balancer sits between the client and service instances and routes requests, hiding instance details from the client.

Retry with Exponential Backoff — On failure, the client waits an exponentially increasing delay before each retry attempt to avoid overwhelming a recovering service.

Idempotent Consumer — A message consumer that safely handles duplicate messages by checking whether a message has already been processed before acting on it.

Kafka — Distributed event streaming platform for asynchronous communication.

RabbitMQ — Message broker used for asynchronous messaging.

Idempotency — Same request produces same result even if repeated multiple times.

OAuth2 — Authorization framework for secure API access.

JWT Token — A compact, self-contained token used for stateless authentication by encoding user claims in a signed JSON structure.

Stateless Service — A service that does not retain any client session data between requests; each request is fully self-contained.

Blue-Green Deployment — Deployment strategy maintaining two identical environments (blue/green) and switching traffic to the new version with zero downtime.

Canary Deployment — Gradually rolling out a new version to a small percentage of users to validate it before full release.

CI/CD — Automated pipeline for continuously integrating code changes, running tests, and deploying to production.

Jenkins — Open-source automation server widely used to create CI/CD pipelines for building, testing, and deploying applications.

Prometheus — Open-source monitoring system that scrapes and stores time-series metrics from services and infrastructure.

Grafana — Visualization tool used to build dashboards and alerts on top of metrics from sources like Prometheus.

OpenAPI/Swagger — Specification and toolset for designing, documenting, and testing RESTful APIs in a standardized way.

Service Mesh — Dedicated infrastructure layer handling service-to-service communication, security, and observability (e.g., Istio).

Istio — Open-source service mesh that manages traffic, enforces policies, and provides observability for Kubernetes-based microservices.

gRPC — High-performance RPC framework using HTTP/2 and Protocol Buffers for efficient, strongly-typed inter-service communication.

CAP Theorem — Distributed systems can guarantee only two of Consistency, Availability, and Partition Tolerance.

Bulkhead Pattern — Isolates failures to prevent entire system impact.

CQRS — Command Query Responsibility Segregation; separates read (query) and write (command) models for scalability and clarity.

Eventual Consistency — Data becomes consistent over time across services.

Docker

Docker Image — A read-only, layered template built from a Dockerfile that contains the application code, runtime, libraries, and configuration needed to create a container.

Docker Container — A running instance of a Docker image; an isolated, lightweight process with its own filesystem, network, and process space.

Dockerfile — A plain-text script containing a sequence of instructions (FROM, RUN, COPY, CMD, etc.) that Docker uses to build an image automatically.

Docker Hub — A public cloud-based registry for storing, sharing, and pulling Docker images; the default registry used by the Docker CLI.

Docker Registry — A storage and distribution system for Docker images; can be public (Docker Hub) or private (AWS ECR, Harbor, GitLab Registry).

docker build — Reads a Dockerfile and builds a new Docker image layer by layer, tagging it with a name and version.

docker run — Creates and starts a new container from a specified image, optionally mapping ports, volumes, and environment variables.

docker pull — Downloads a Docker image from a registry to the local machine without running it.

docker push — Uploads a locally built Docker image to a remote registry for sharing or deployment.

docker ps — Lists all currently running containers; use `docker ps -a` to include stopped containers.

docker stop — Gracefully stops a running container by sending a SIGTERM signal followed by SIGKILL if it does not stop in time.

docker rm — Removes one or more stopped containers from the local system.

docker rmi — Deletes one or more Docker images from the local image cache.

docker exec — Runs a command inside an already running container, commonly used to open an interactive shell for debugging.

docker logs — Fetches and displays the stdout/stderr logs output by a running or stopped container.

docker inspect — Returns detailed low-level JSON information about a container, image, or network including configuration and state.

docker images — Lists all Docker images stored locally along with their tag, image ID, and size.

docker-compose — A tool for defining and running multi-container Docker applications using a docker-compose.yml file with a single command.

docker-compose.yml — A YAML file that describes the services, networks, and volumes for a multi-container application managed by Docker Compose.

docker-compose up — Builds, creates, and starts all containers defined in the docker-compose.yml file in the correct dependency order.

docker-compose down — Stops and removes all containers, networks, and optionally volumes defined in the docker-compose.yml file.

Layer — Each instruction in a Dockerfile creates an immutable read-only layer; Docker caches layers to speed up subsequent builds.

Layer Caching — Docker reuses unchanged layers from previous builds, significantly reducing build time by only rebuilding layers that changed.

Base Image — The starting image specified in the FROM instruction of a Dockerfile on which all subsequent layers are stacked.

Multi-Stage Build — A Dockerfile technique using multiple FROM instructions to separate build and runtime stages, producing a smaller, cleaner final image.

Entrypoint — The ENTRYPOINT instruction in a Dockerfile defines the executable that always runs when the container starts; arguments from CMD are appended.

CMD — Provides default arguments or the default command to run in a container; can be overridden at docker run time.

EXPOSE — Documents which port the containerized application listens on at runtime; does not publish the port to the host.

Port Mapping (-p) — Maps a host machine port to a container port so external traffic can reach the containerized application.

Volume — A persistent, managed storage unit that exists outside the container's filesystem and survives container restarts and deletions.

Bind Mount — Mounts a specific directory from the host machine directly into the container, useful for development with live code reloading.

Docker Network — A virtual network that allows containers to communicate with each other securely and in isolation from the host or other networks.

Bridge Network — Default Docker network driver that creates an isolated internal network where containers on the same host can communicate.

Host Network — A network mode where the container shares the host machine's network stack directly, with no network isolation.

Overlay Network — Docker network driver that enables containers running on different Docker hosts (in a Swarm) to communicate securely.

ENV — Dockerfile instruction that sets environment variables inside the image, accessible to the application at runtime.

ARG — Dockerfile instruction that defines build-time variables passed with --build-arg; not available in the running container.

Docker Swarm — Docker's built-in native clustering and orchestration tool for deploying and managing services across multiple Docker hosts.

Docker Desktop — A GUI application for macOS and Windows that bundles Docker Engine, Docker CLI, Docker Compose, and Kubernetes in one install.

Docker Context — A configuration that tells the Docker CLI which Docker Engine to connect to (local, remote, or cloud-based).

Healthcheck — A Dockerfile or Compose instruction that defines a command Docker runs periodically to verify the container is still healthy.

Container Orchestration — Automated management of container lifecycle including scheduling, scaling, networking, and self-healing across a cluster.

ECR (Elastic Container Registry) — AWS's fully managed private Docker container registry for storing, managing, and deploying container images securely.

Image Tag — A label (e.g., latest, v1.0.0) appended to an image name to identify a specific version of that image.

docker tag — Creates an additional tag/alias for an existing local image, typically used before pushing to a registry with a new name or version.

docker network ls — Lists all Docker networks currently configured on the host.

docker volume ls — Lists all Docker volumes currently created on the host.

docker system prune — Removes all stopped containers, unused networks, dangling images, and optionally unused volumes to free disk space.

Dangling Image — An untagged image layer that is no longer referenced by any named image, typically left over from builds; safely removable.

Container Lifecycle — The sequence of states a container passes through: Created, Running, Paused, Stopped, and Removed.

Immutable Infrastructure — A deployment philosophy where containers are never modified after deployment; changes are made by replacing containers with new images.

Docker Daemon — The background service (dockerd) running on the host that manages images, containers, networks, and volumes via the Docker API.

Docker CLI — The command-line client (docker) that communicates with the Docker Daemon via REST API to execute user commands.

Namespace — Linux kernel feature used by Docker to isolate container processes, file systems, network stacks, and user IDs from each other and the host.

cgroups (Control Groups) — Linux kernel feature used by Docker to limit and monitor CPU, memory, and I/O resource usage per container.

Union File System — A layered filesystem (e.g., OverlayFS) that Docker uses to stack image layers efficiently without duplicating data.

Performance / Optimization

Performance Optimization — Improving application speed, scalability, and resource usage.

Scalability — Ability to handle increased load efficiently.

Latency — Time taken to process single request.

Throughput — Number of requests processed per second.

Bottleneck — Component limiting system performance.

Profiling — Analyzing application performance and resource usage.

CPU Bound Task — Task mainly limited by CPU processing power.

IO Bound Task — Task mainly waiting for I/O operations.

Memory Leak — Unused objects retained in memory unnecessarily.

Garbage Collection (GC) — Automatic memory cleanup for unused objects.

Minor GC — Cleans young generation memory.

Major GC — Cleans old generation memory.

Full GC — Cleans entire heap causing application pause.

Heap Memory — JVM memory storing objects.

Stack Memory — JVM memory storing method calls and local variables.

JVM Tuning — Adjusting JVM settings for better performance.

G1 GC — Low-pause garbage collector for large heap applications.

ZGC — Ultra-low latency scalable garbage collector.

CMS GC — Concurrent collector minimizing pause times.

JIT Compiler — Converts bytecode into optimized native machine code.

Escape Analysis — JVM optimization deciding object allocation location.

Object Pooling — Reusing objects instead of creating repeatedly.

Connection Pooling — Reusing database/network connections efficiently.

HikariCP — High-performance JDBC connection pooling library.

Caching — Storing frequently accessed data for faster retrieval.

In-Memory Cache — Cache stored directly in RAM.

Redis Cache — Distributed in-memory caching datastore.

EhCache — Java-based caching framework.

Lazy Loading — Loads data only when required.

Eager Loading — Loads related data immediately.

Pagination — Fetching large data in smaller chunks.

Batch Processing — Processing multiple records together efficiently.

Batch Insert — Inserting multiple DB records in single operation.

Indexing — Improving query speed using indexes.

Composite Index — Index created on multiple columns.

Covering Index — Index containing all required query columns.

Query Optimization — Improving database query execution efficiency.

Execution Plan — Database strategy used to execute query.

N+1 Query Problem — Excessive DB queries caused by repeated fetching.

Denormalization — Adding redundancy to improve read performance.

Horizontal Scaling — Adding more servers to distribute load.

Vertical Scaling — Increasing server CPU/RAM resources.

Load Balancing — Distributing requests across multiple servers.

Asynchronous Processing — Executing tasks without blocking main thread.

Multithreading — Running multiple threads concurrently.

Parallel Processing — Executing tasks simultaneously on multiple cores.

Thread Pooling — Reusing worker threads instead of creating repeatedly.

Lock Contention — Multiple threads competing for same lock.
Deadlock — Threads waiting indefinitely for each other's locks.
Thread Safety — Correct behavior under concurrent execution.
Immutable Object — Object whose state cannot change after creation.
Serialization Cost — Performance overhead during object conversion.
Compression — Reducing data size for faster transfer/storage.
CDN — Delivers static content from geographically nearest servers.
Rate Limiting — Restricting request frequency to protect system.
Backpressure — Slowing incoming requests during overload.
Circuit Breaker — Stops repeated failing service calls temporarily.
Bulkhead Pattern — Isolates failures between components/resources.
Debouncing — Delays repeated event execution until idle period.
Throttling — Limits execution frequency of repeated events.
Streaming — Processing data continuously instead of loading entirely.
Memory Optimization — Reducing unnecessary memory consumption.
Hotspot — Frequently executed code area targeted for optimization.
Cold Start — Initial startup delay before application becomes responsive.
Warmup Phase — JVM optimization period before peak performance achieved.
Benchmarking — Measuring system/application performance scientifically.
Load Testing — Evaluating behavior under heavy traffic/load.
Stress Testing — Testing beyond expected load capacity.
Concurrency — Managing multiple tasks simultaneously.
Parallelism — Executing multiple tasks literally at same time.
Sharding — Splitting database across multiple servers.
Replication — Maintaining copies of data across servers.
Event-Driven Architecture — Systems communicating using asynchronous events.
Kafka — High-throughput distributed event streaming platform.
Reactive Programming — Asynchronous non-blocking programming model.
WebFlux — Spring reactive framework for non-blocking APIs.
Non-Blocking IO — IO operations not blocking executing thread.
Blocking IO — Thread waits until IO operation completes.

System Design

System Design — Designing scalable, reliable, and maintainable software systems.
Scalability — Ability of system to handle increasing load efficiently.
High Availability — System remains operational with minimal downtime.
Reliability — System consistently performs expected operations correctly.
Fault Tolerance — System continues functioning even after failures.
Latency — Time taken to process request and return response.
Throughput — Number of requests processed per second.
Horizontal Scaling — Adding more servers to distribute load.
Vertical Scaling — Increasing CPU/RAM of existing server.
Load Balancer — Distributes traffic across multiple servers.
Reverse Proxy — Server handling client requests before backend services.
API Gateway — Single entry point managing APIs and routing.
Rate Limiting — Restricting request frequency to prevent abuse.
Caching — Storing frequently accessed data for faster retrieval.
Cache Hit — Requested data found in cache.
Cache Miss — Requested data not found in cache.
Redis — In-memory datastore commonly used for caching.
CDN — Delivers content from geographically closer servers.
Database Sharding — Splitting database across multiple servers.
Partitioning — Dividing large dataset into smaller parts.
Replication — Maintaining copies of data across servers.
Master-Slave Replication — Primary DB handles writes while replicas handle reads.
Read Replica — Secondary database serving read operations.
Consistency — All users see same latest data.
Eventual Consistency — Data becomes consistent over time.
CAP Theorem — Distributed systems guarantee only two among Consistency, Availability, Partition Tolerance.
ACID — Transaction guarantees for relational databases.
BASE — Basically Available, Soft state, Eventual consistency model.

SQL Database — Relational database with structured schema.

NoSQL Database — Flexible schema database optimized for scalability.

Document Database — NoSQL DB storing JSON-like documents.

Key-Value Store — NoSQL DB storing key-value pairs.

Columnar Database — Database optimized for analytical queries.

Graph Database — Database optimized for relationship-based queries.

Microservices — Architecture splitting application into independent services.

Monolith — Single tightly coupled application architecture.

Service Discovery — Dynamic detection of service locations.

Circuit Breaker — Prevents cascading failures between services.

Retry Mechanism — Automatically retries failed operations.

Bulkhead Pattern — Isolates failures between components/resources.

Saga Pattern — Handles distributed transactions using compensating actions.

CQRS — Separates read and write operations.

Event Sourcing — Stores state changes as events.

Message Queue — Enables asynchronous communication between systems.

Kafka — Distributed event streaming platform.

RabbitMQ — Message broker for asynchronous messaging.

Pub/Sub Model — Publishers send events to multiple subscribers.

Producer — Service sending messages/events.

Consumer — Service processing messages/events.

Idempotency — Same request produces same result repeatedly.

Distributed Lock — Lock mechanism across distributed systems.

Sticky Session — Client routed repeatedly to same server.

Stateless Service — Service not storing session data locally.

Stateful Service — Service maintaining client/session state.

Health Check — Mechanism checking service availability.

Heartbeat — Periodic signal confirming server/service alive status.

Failover — Switching to backup system during failure.

Disaster Recovery — Restoring systems after catastrophic failure.

Blue-Green Deployment — Switching traffic between old and new environments.

Canary Deployment — Gradually releasing new version to limited users.

Rolling Deployment — Updating servers incrementally without downtime.

CI/CD — Automated build, test, and deployment pipeline.

Docker — Containerization platform packaging app with dependencies.

Containerization — Packaging application and runtime together.

Kubernetes — Container orchestration platform for scaling/deployment.

Pod — Smallest deployable unit in Kubernetes.

Ingress — Routes external traffic into Kubernetes services.

Observability — Monitoring internal system state using logs/metrics/traces.

Logging — Recording system/application events.

Monitoring — Tracking system health and performance metrics.

Alerting — Notifying teams about failures or threshold breaches.

Distributed Tracing — Tracking requests across multiple services.

Prometheus — Metrics collection and monitoring tool.

Grafana — Dashboard visualization tool for metrics.

ELK Stack — Elasticsearch, Logstash, Kibana for centralized logging.

SLA — Service Level Agreement defining expected uptime/performance.

SLO — Internal reliability/performance target.

SLI — Measurement metric for service performance.

Hotspot — Overloaded server/resource causing bottleneck.

Bottleneck — Component limiting system performance.

Backpressure — System slowing incoming traffic during overload.

Throttling — Intentionally limiting request processing rate.

Autoscaling — Automatically increasing/decreasing resources based on load.

Long Polling — Client waits until server has data available.

WebSocket — Full-duplex persistent communication protocol.

Polling — Client repeatedly checking server for updates.

DNS — Converts domain names into IP addresses.

TLS/SSL — Encryption protocol securing network communication.

OAuth2 — Authorization framework for secure API access.

JWT — Stateless token-based authentication mechanism.

Zero Downtime Deployment — Deploying application without service interruption.

Data Lake — Centralized storage for raw structured/unstructured data.

Data Warehouse — Optimized storage for analytics/reporting.

OLTP — Online transactional processing systems.

OLAP — Analytical processing systems for reporting.

Design Pattern

Creational Pattern — Pattern dealing with object creation mechanisms.

Structural Pattern — Pattern focusing on class/object composition.

Behavioral Pattern — Pattern defining communication between objects.

Singleton Pattern — Ensures only one object instance exists globally.

Factory Pattern — Creates objects without exposing creation logic.

Abstract Factory Pattern — Creates families of related objects together.

Builder Pattern — Constructs complex objects step by step.

Prototype Pattern — Creates objects by cloning existing instances.

Object Pool Pattern — Reuses objects instead of creating repeatedly.

Adapter Pattern — Converts incompatible interfaces into compatible ones.

Bridge Pattern — Separates abstraction from implementation independently.

Composite Pattern — Treats individual objects and groups uniformly.

Decorator Pattern — Adds functionality dynamically without modifying object.

Facade Pattern — Provides simplified interface to complex subsystem.

Flyweight Pattern — Reduces memory usage by sharing objects.

Proxy Pattern — Controls access to another object.

Observer Pattern — One object notifies multiple dependent objects automatically.

Strategy Pattern — Encapsulates interchangeable algorithms separately.

Command Pattern — Encapsulates request as object.

State Pattern — Changes object behavior based on internal state.

Template Method Pattern — Defines algorithm skeleton with customizable steps.

Chain of Responsibility Pattern — Passes request through handler chain sequentially.

Mediator Pattern — Centralizes communication between multiple objects.

Memento Pattern — Captures and restores object state.

Iterator Pattern — Sequentially accesses collection elements without exposing structure.

Visitor Pattern — Adds operations to objects without modifying them.

Interpreter Pattern — Defines grammar and interprets language expressions.

IOC Pattern — Spring container controls object creation and lifecycle.

Dependency Injection Pattern — Dependencies injected externally into objects.

MVC Pattern — Separates application into Model, View, and Controller layers.

DAO Pattern — Separates persistence logic from business logic.

DTO Pattern — Transfers data between application layers efficiently.

Repository Pattern — Encapsulates database access operations.

Front Controller Pattern — Single controller handling all incoming requests.

Service Locator Pattern — Central registry providing service lookup.

Microkernel Pattern — Core system extended using plugins/modules.

Circuit Breaker Pattern — Prevents cascading failures in distributed systems.

Saga Pattern — Handles distributed transactions using compensating actions.

CQRS Pattern — Separates read and write operations.

Event Sourcing Pattern — Stores application state as sequence of events.

Retry Pattern — Automatically retries failed operations temporarily.

Bulkhead Pattern — Isolates failures between components/resources.

Sidecar Pattern — Helper service/container running alongside main application.

Strangler Pattern — Gradually replaces legacy system with modern services.

Anti-Corruption Layer Pattern — Protects new system from legacy system complexity.

Frequently Asked Cross Questions

Q: Why Singleton can be dangerous?

Shared mutable state may cause threading and testing issues.

Q: Difference between Factory and Abstract Factory?

Factory creates one product type; Abstract Factory creates related product families.

Q: Difference between Strategy and State Pattern?

Strategy changes algorithm externally; State changes behavior internally.

Q: Difference between Adapter and Decorator?

Adapter changes interface; Decorator adds functionality.

Q: Most used design patterns in Spring?

Singleton, Factory, Proxy, MVC, Dependency Injection, Template Method.

Q: Which pattern is used in Spring Bean creation?

Factory and Singleton patterns.

Q: Which pattern is used in Spring AOP?

Proxy pattern.

Q: Which pattern is used in RestTemplate/JdbcTemplate?

Template Method pattern.

Java 8

Lambda Expression — Anonymous function enabling concise functional-style code.

Functional Interface — Interface containing exactly one abstract method.

@FunctionalInterface — Annotation ensuring interface has one abstract method only.

Predicate — Functional interface testing condition and returning boolean.

Function — Functional interface transforming input into output.

Consumer — Functional interface accepting input without returning value.

Supplier — Functional interface supplying values without input.

Method Reference — Shorter syntax referring existing methods using ::.

Stream API — A powerful API for processing sequences of elements in a functional, declarative style with lazy evaluation.

Intermediate Operation — A Stream operation (e.g., filter, map) that returns a new Stream and is evaluated lazily.

Terminal Operation — A Stream operation (e.g., collect, forEach) that triggers the pipeline and produces a final result.

filter() — Selects elements matching condition.

map() — Intermediate operation that transforms each element into another form using a provided Function.

flatMap() — Intermediate operation that flattens a stream of streams (or collections) into a single stream.

reduce() — Terminal operation that combines all stream elements into a single result using an accumulator function.

collect() — Terminal operation that converts stream elements into a collection or other container using a Collector.

forEach() — Iterates through stream elements.

distinct() — Removes duplicate elements from stream.

sorted() — Intermediate operation that returns a stream with elements sorted in natural or custom order.

limit() — Restricts number of stream elements.

skip() — Skips specified number of elements.

findFirst() — Returns first matching element.

findAny() — Returns any matching element.

anyMatch() — Checks if any element matches condition.

allMatch() — Checks if all elements match condition.

noneMatch() — Checks if no elements match condition.

Optional Class — Container avoiding NullPointerException.

Optional.of() — Creates an Optional with a non-null value; throws NullPointerException if the value is null.

Optional.ofNullable() — Creates an Optional that may hold a null value without throwing an exception.

Optional.empty() — Creates empty Optional object.

isPresent() — Checks if Optional contains value.

orElse() — Returns default value if Optional empty.

orElseGet() — Lazily returns default value using Supplier.

orElseThrow() — Throws exception if Optional empty.

ifPresent() — Executes logic if Optional contains value.

Default Method — Interface method with implementation.

Static Method in Interface — Utility method defined inside interface.

Date and Time API — Improved immutable date/time handling introduced in Java 8.

LocalDate — Represents date without time.

LocalTime — Represents time without date.

LocalDateTime — Represents both date and time together without timezone information.

ZonedDateTime — Represents a full date, time, and timezone, useful for timezone-aware applications.

Instant — Represents timestamp in UTC.

Parallel Stream — Processes stream elements concurrently using multiple threads.

Sequential Stream — Processes elements one by one.

Collectors — Utility class for collecting stream results.

groupingBy() — Groups stream elements based on condition/key.

partitioningBy() — Splits elements into true/false groups.

joining() — Collector that concatenates stream elements into a single String with optional delimiter, prefix, and suffix.

counting() — Collector that counts the number of elements in the stream.

mapping() — Collector that applies a mapping function before collecting into another collector.

min() — Terminal operation that returns the minimum element according to a Comparator, wrapped in Optional.

max() — Terminal operation that returns the maximum element according to a Comparator, wrapped in Optional.

sum() — Terminal operation on numeric streams (IntStream, LongStream, DoubleStream) that calculates the total of all elements.

Stream Pipeline — A chain of source, intermediate operations, and a terminal operation executed as a single unit lazily.

Lazy Evaluation — Intermediate operations in a stream are not executed until a terminal operation is invoked.

Short Circuiting — A stream optimization where processing stops as soon as the result can be determined (e.g., findFirst, anyMatch).

Comparator.comparing() — Simplifies object sorting using fields.

forEachOrdered() — Maintains encounter order during parallel stream processing.

peek() — Performs action during stream processing mainly for debugging.

Double Colon Operator (::) — Syntax used for method references.

Nashorn Engine — JavaScript engine introduced in Java 8.

CompletableFuture — Provides a rich API for composing asynchronous computations with methods like thenApply, thenCompose, and exceptionally.

Base64 Encoder/Decoder — Utility for Base64 encoding and decoding.

Splitter — A special iterator introduced in Java 8 designed to traverse and partition stream elements for parallel processing.

Type Inference — Compiler automatically determines datatype.

Effectively Final Variable — Variable whose value does not change after initialization.

Immutable Date API — Java 8 date/time objects are immutable and thread-safe.

Functional Programming — Programming style using functions and immutability.

Internal Iteration — Stream API controls iteration internally.

External Iteration — Programmer manually controls iteration using loops.

Spring

IOC (Inversion of Control) — Design principle where the framework controls object creation and lifecycle instead of the developer.

Dependency Injection (DI) — Injecting dependencies externally instead of creating manually.

Bean — An object created and managed by the Spring IoC container, configured via annotations or XML.

ApplicationContext — Advanced Spring IoC container that extends BeanFactory with event publishing, internationalization, and AOP support.

BeanFactory — The simplest Spring IoC container providing basic bean creation and dependency injection without advanced features.

Spring Container — The runtime environment that instantiates, configures, wires, and manages the lifecycle of Spring beans.

@Component — Generic stereotype annotation marking a class as a Spring-managed component eligible for auto-detection.

@Service — Specialization of @Component used to annotate business logic classes, aiding in readability and layering.

@Repository — Specialization of @Component for DAO classes; also enables automatic Spring data access exception translation.

@Controller — Specialization of @Component for Spring MVC controllers that handle incoming web requests.

@RestController — Combination of @Controller and @ResponseBody; all methods return data directly as HTTP response body.

@Autowired — Instructs Spring to automatically resolve and inject a bean dependency by matching type.

Constructor Injection — Preferred injection style where dependencies are provided through the constructor, enabling immutability.

Setter Injection — Optional injection style where dependencies are injected via public setter methods after object construction.

Field Injection — Injects dependencies directly into fields using @Autowired, though discouraged for production code.

Loose Coupling — Design where components depend on abstractions (interfaces) rather than concrete classes, easing replacement.

Tight Coupling — Design where components are directly dependent on concrete implementations, making changes difficult.

@Qualifier — Resolves bean ambiguity when multiple beans exist.

@Primary — Marks one bean as the preferred default when multiple beans of the same type exist in the context.

@Bean — Method-level annotation in a @Configuration class telling Spring to manage the returned object as a bean.

@Configuration — Class-level annotation declaring that the class contains Spring bean definitions.

Singleton Scope — Default Spring bean scope; only one instance of the bean is created and shared across the entire application.

Prototype Scope — A new bean instance is created every time the bean is requested from the Spring container.

Request Scope — A new bean instance is created for every HTTP request; only available in web-aware contexts.

Session Scope — A new bean instance is created for every HTTP session; only available in web-aware contexts.

Bean Lifecycle — The sequence of events from bean instantiation, dependency injection, initialization, use, to destruction.

InitializingBean — Interface whose `afterPropertiesSet()` method is called by Spring after all properties are set.

DisposableBean — Interface whose `destroy()` method is called by Spring before the bean is removed from the container.

Spring AOP — Aspect-Oriented Programming support in Spring for modularizing cross-cutting concerns like logging and security.

Aspect — A module encapsulating cross-cutting concerns (e.g., logging, transaction management) separate from business logic.

Advice — The action taken by an aspect at a particular join point (e.g., before, after, or around method execution).

Join Point — A specific point during execution (e.g., a method call) where an aspect's advice can be applied.

Pointcut — An expression that matches one or more join points, defining where advice should be applied.

Before Advice — Advice that executes before the matched method is invoked, without the ability to prevent it.

After Advice — Advice that executes after the matched method completes (whether normally or with an exception).

Around Advice — Most powerful advice type that wraps method execution and can control whether the method runs at all.

Spring MVC — Web framework implementing Model-View-Controller pattern.

DispatcherServlet — Central front controller in Spring MVC that receives all requests and delegates to the appropriate handler.

Model — Holds application/business data.

View — UI representation of application data.

Controller — Handles user requests and responses.

Handler Mapping — Maps incoming request URLs to the correct controller method based on annotations or configuration.

View Resolver — Translates a logical view name returned by a controller into an actual view resource (e.g., JSP, Thymeleaf).

Spring JDBC — Spring module that simplifies plain JDBC code by removing repetitive boilerplate around connections and exceptions.

JdbcTemplate — Helper class reducing JDBC boilerplate code.

Transaction Management — Spring's ability to manage ACID database transactions declaratively or programmatically.

@Transactional — Declarative annotation for wrapping methods in a Spring-managed transaction with rollback support.

Propagation — Defines how transactions behave when one transactional method calls another (e.g., REQUIRED, REQUIRES_NEW).

Isolation Level — Defines how visible uncommitted changes of one transaction are to other concurrent transactions.

Spring ORM — Integration module providing support for JPA, Hibernate, and JDO within the Spring ecosystem.

Spring Security — Framework for authentication and authorization.

Authentication — The process of verifying who a user is (e.g., validating username and password).

Authorization — The process of determining what an authenticated user is allowed to do or access.

JWT Authentication — Stateless authentication mechanism where a signed JWT token is passed with each request, avoiding server-side sessions.

CSRF — Cross-Site Request Forgery attack where a malicious site tricks a logged-in user's browser into making an unwanted request.

Spring Batch — Framework for building robust batch processing jobs with features like chunking, retry, skip, and restart.

Spring Integration — Framework for building enterprise integration patterns like messaging, routing, and transformation.

Spring Cloud — Collection of tools for microservices patterns including distributed configuration, service discovery, and circuit breakers.

Spring Boot — Simplifies Spring application setup with auto configuration.

Spring Profiles — Environment-specific bean/configuration management.

@Profile — Activates beans for specific environment.

Property Source — Externalized configuration source.

Environment Object — Spring abstraction providing access to properties from `application.properties`, environment variables, and profiles.

Event Listener — Handles application events asynchronously/synchronously.

@EventListener — Marks a method to be invoked when a specific application event is published in the context.

REST API — Architectural style using HTTP verbs (GET, POST, PUT, DELETE) and stateless requests for web service communication.

SOAP — XML-based messaging protocol for exchanging structured information in web services over HTTP or other protocols.

Interceptor — A Spring component (`HandlerInterceptor`) that intercepts HTTP requests before and after controller processing.

Filter — A Java Servlet component that intercepts requests and responses before they reach the Spring `DispatcherServlet`.

Exception Handling — Managing application errors gracefully.

@ControllerAdvice — Defines a global exception handler, data binder, or model attribute provider across all controllers.

@ExceptionHandler — Inside `@ControllerAdvice` or a controller, handles a specific exception and returns a custom response.

Spring Cache — Abstraction layer that transparently applies caching to method results using providers like Redis or EhCache.

@Cacheable — Intercepts a method call and returns a cached result if available; invokes the method and caches on first call.

Scheduling — Running tasks at defined intervals or cron expressions using Spring's built-in task scheduling support.

@Scheduled — Triggers a Spring-managed method at fixed rate, fixed delay, or cron expression automatically.

Async Processing — Executing long-running or I/O-heavy tasks in background threads without blocking the main thread.

@Async — Tells Spring to execute the annotated method asynchronously in a separate thread from a TaskExecutor.

SpringBoot

Spring Boot — Framework used to build production-ready Spring applications quickly with minimal configuration.

Starter Dependency — Predefined dependency bundle simplifying project setup.

Auto Configuration — Automatically configures Spring application based on dependencies present.

Embedded Server — Built-in Tomcat/Jetty server eliminating external deployment need.

Spring Initializr — A web-based tool (start.spring.io) for generating a Spring Boot project with chosen dependencies and structure.

application.properties — The default configuration file where application settings like port, DB URL, and logging are defined.

application.yml — YAML-format alternative to application.properties offering cleaner hierarchical configuration syntax.

@SpringBootApplication — Composite annotation combining @Configuration, @ComponentScan, and @EnableAutoConfiguration on the main class.

@RestController — Combines @Controller and @ResponseBody, making every method return JSON/XML directly as HTTP response.

@Controller — Marks a class as a Spring MVC controller that handles web requests and returns view names.

@Service — Marks a class as a business logic/service layer component managed by the Spring container.

@Repository — Marks a class as a data access layer component and enables Spring's persistence exception translation.

@Component — Generic stereotype annotation that marks any class as a Spring-managed bean.

@Autowired — Instructs Spring to automatically inject the matching dependency by type into the annotated field, constructor, or setter.

Dependency Injection — A design pattern where an object's dependencies are provided externally by the Spring container rather than created internally.

IOC Container — The core Spring container that manages the full lifecycle of beans, including creation, wiring, and destruction.

Bean — Any object that is instantiated, assembled, and managed by the Spring IOC container.

@Bean — Annotates a method in a @Configuration class to indicate its return value should be registered as a Spring bean.

@Configuration — Marks a class as a source of bean definitions; equivalent to XML-based Spring configuration.

@Value — Injects a value from application.properties or environment variables into a field or method parameter.

@Qualifier — Used alongside @Autowired to specify which bean to inject when multiple candidates of the same type exist.

@Primary — Marks one bean as the default choice when multiple beans of the same type are available for injection.

@RequestMapping — Maps HTTP requests to a specific handler class or method based on URL, method, headers, or parameters.

@GetMapping — Handles HTTP GET requests.

@PostMapping — Handles HTTP POST requests.

@PutMapping — Handles HTTP PUT requests.

@DeleteMapping — Handles HTTP DELETE requests.

@PathVariable — Extracts a dynamic value from a (URL path) URI template variable and binds it to a method parameter.

@RequestParam — Binds a query string parameter from the request URL to a method parameter.

@RequestBody — Deserializes the HTTP request body (typically JSON) into a Java object using Jackson.

@ResponseBody — Serializes the return value of a controller method into the HTTP response body as JSON/XML.

Spring Data JPA — Abstraction layer on top of JPA/Hibernate that auto-generates common DB operations from repository interfaces.

JpaRepository — Extends CrudRepository with JPA-specific methods like flush(), saveAndFlush(), and findAll(Pageable).

Hibernate — ORM framework mapping Java objects to database tables.

ORM — Object-Relational Mapping; a technique for converting data between Java objects and relational database tables.

Entity — A Java class annotated with @Entity that is mapped to a database table by JPA/Hibernate.

@Entity — Marks a Java class as a JPA entity that will be mapped to a database table.

@Table — Specifies the name of the database table that the entity class maps to.

@Id — Marks a field as the primary key of the entity.

@GeneratedValue — Configures automatic primary key generation strategy (AUTO, IDENTITY, SEQUENCE, TABLE).

@Transactional — Wraps a method or class in a database transaction; rolls back on unchecked exceptions by default.

Spring Security — Framework for authentication and authorization.

JWT — JSON Web Token; a compact, URL-safe token format for stateless authentication, containing header, payload, and signature.

Actuator — Spring Boot module exposing production-ready endpoints for health checks, metrics, environment info, and more.

Profile — A mechanism for defining environment-specific bean configurations (e.g., dev, test, prod).

@Profile — Activates a bean or configuration class only when the specified profile is active.

Exception Handling — Managing application errors gracefully.

@ControllerAdvice — Enables centralized exception handling across all controllers in a Spring application.

@ExceptionHandler — Defines a method to handle a specific exception type and return a custom response.

Pagination — Technique for returning data in pages using Pageable and Page objects to avoid loading large datasets at once.

Caching — Storing frequently accessed method results temporarily to reduce redundant DB queries and improve performance.

@Cacheable — Caches the result of a method call; subsequent calls with the same arguments return the cached result.

Spring Batch — Framework for processing large volumes of records in batch jobs with built-in retry, skip, and restart support.

Spring Scheduler — Built-in scheduling support for running methods at fixed intervals or cron expressions.

@Scheduled — Triggers a method execution at a fixed rate, fixed delay, or cron expression without external scheduling tools.

Spring Cloud — Umbrella project providing tools for distributed system patterns like config management, service discovery, and circuit breaking.

Feign Client — Declarative REST client that generates HTTP client code from an annotated interface, simplifying service-to-service calls.

Eureka — Netflix service registry integrated with Spring Cloud for microservice registration and discovery.

Config Server — Spring Cloud component that serves externalized configuration from a central Git repository to all microservices.

Circuit Breaker — Pattern implemented via Resilience4j or Hystrix to stop calling failing services and return fallback responses.

Resilience4j — Lightweight fault-tolerance library providing circuit breaker, retry, rate limiter, and time limiter for Spring Boot.

Swagger/OpenAPI — Integration that auto-generates interactive API documentation from controller annotations at runtime.

Lombok — Java library that auto-generates boilerplate code like getters, setters, constructors, and builders via annotations.

DevTools — Spring Boot module that enables automatic application restart and LiveReload on classpath changes during development.

CommandLineRunner — Functional interface whose run() method is called once the Spring Boot application context is fully started.

WAR Packaging — Packaging format used when deploying a Spring Boot app to an external servlet container like Tomcat.

JAR Packaging — Default Spring Boot packaging format that bundles the app and embedded server into a single executable JAR.

Spring Security

SecurityFilterChain — The primary Spring Security 6+ configuration bean that defines the HTTP security rules, replacing the deprecated WebSecurityConfigurerAdapter.

WebSecurityConfigurerAdapter — The older (deprecated since Spring Security 5.7) base class overridden to customize HTTP security, authentication, and authorization rules.

HttpSecurity — A builder API used inside SecurityFilterChain to configure URL access rules, login/logout, CSRF, CORS, session management, and filter chains.

Filter Chain — A sequence of security filters (e.g., UsernamePasswordAuthenticationFilter, JwtFilter) that every HTTP request passes through before reaching the controller.

OncePerRequestFilter — A base class for writing a custom security filter that guarantees it executes exactly once per HTTP request, commonly used for JWT validation.

UsernamePasswordAuthenticationFilter — A built-in Spring Security filter that intercepts login requests, extracts credentials, and delegates to the AuthenticationManager.

AuthenticationManager — The core interface responsible for processing an Authentication object and returning a fully authenticated object or throwing an exception.

AuthenticationProvider — A strategy interface that performs a specific authentication mechanism (e.g., DaoAuthenticationProvider for username/password).

DaoAuthenticationProvider — The default AuthenticationProvider that loads user details from a UserDetailsService and validates the password using a PasswordEncoder.

UserDetailsService — A core interface with a single method loadUserByUsername() that fetches user data from a data source for authentication.

UserDetails — An interface representing a user with fields for username, password, authorities, and account status flags used by Spring Security.

UserDetailsImpl — A common custom implementation of UserDetails that maps application User entity fields to the Spring Security user model.

GrantedAuthority — An interface representing a permission or role (e.g., ROLE_ADMIN, ROLE_USER) granted to an authenticated user.

SecurityContext — A holder for the current user's authentication details (Authentication object) that is accessible throughout the request lifecycle.

SecurityContextHolder — A static class that stores and provides access to the SecurityContext; default strategy is thread-local storage per request.

Authentication Object — An interface holding the principal (user), credentials (password), and granted authorities of the currently authenticated user.

Principal — The identity of the currently authenticated user, typically the UserDetails object or username string stored in the Authentication object.

PasswordEncoder — An interface for encoding and verifying passwords; BCryptPasswordEncoder is the recommended implementation for secure hashing.

BCryptPasswordEncoder — Spring Security's recommended password encoder that uses the BCrypt adaptive hashing algorithm with salt, making brute-force attacks computationally expensive.

JWT Header — The first part of a JWT token encoded in Base64, containing the algorithm (e.g., HS256) and token type.

JWT Payload (Claims) — The second part of a JWT token containing the claims: subject (username), issued-at, expiration, roles, and any custom data.

JWT Signature — The third part of a JWT token created by signing the encoded header and payload with a secret key to verify integrity.

JWT Secret Key — A private key (or secret string) used to sign and verify JWT tokens; must be kept confidential and rotated periodically.

JWT Expiration — A claim (exp) in the JWT payload defining the timestamp after which the token is no longer valid and must be rejected.

Refresh Token — A long-lived token issued alongside the access JWT, used to obtain a new access token after it expires without re-authenticating.

Access Token — A short-lived JWT sent with every API request in the Authorization header to prove the user's identity and permissions.

Bearer Token — An authentication scheme where the client sends the JWT in the HTTP header as `Authorization: Bearer <token>`.

Stateless Authentication — An approach where no session is stored on the server; every request is self-authenticated using the JWT it carries.

OAuth2 — An open authorization framework that allows third-party applications to obtain limited access to a user's account without exposing credentials.

OAuth2 Resource Server — A Spring Security configuration that validates incoming JWT Bearer tokens and protects API endpoints from unauthorized access.

OAuth2 Authorization Server — The server responsible for authenticating users and issuing access tokens, refresh tokens, and authorization codes.

OAuth2 Client — An application that requests access to a protected resource on behalf of a user using tokens obtained from the authorization server.

Authorization Code Flow — The most secure OAuth2 flow where the client receives an authorization code via redirect and exchanges it for tokens server-side.

Client Credentials Flow — An OAuth2 flow for machine-to-machine communication where the client authenticates using its own ID and secret, not a user.

Implicit Flow — A deprecated OAuth2 flow where tokens were returned directly in the URL fragment; replaced by Authorization Code with PKCE.

PKCE (Proof Key for Code Exchange) — An OAuth2 extension that prevents authorization code interception attacks in public clients like SPAs and mobile apps.

OpenID Connect (OIDC) — An identity layer on top of OAuth2 that returns an ID token (JWT) with user profile information alongside the access token.

ID Token — A JWT issued by an OIDC provider containing identity claims (name, email, sub) about the authenticated user.

Scope — A parameter in OAuth2 that defines the level of access requested by the client (e.g., read, write, openid, email, profile).

Keycloak — An open-source identity and access management server that provides OAuth2, OIDC, SAML, SSO, and user federation out of the box.

Spring OAuth2 Login — A Spring Security feature that enables social or enterprise login via OAuth2/OIDC providers like Google, GitHub, or Okta.

CSRF Token — A unique, unpredictable token included in every state-changing request to verify it originated from the legitimate application, preventing CSRF attacks.

CSRF Protection — Spring Security enables CSRF protection by default for web apps; it should be disabled for stateless REST APIs using JWT.

CORS Configuration — Spring Security's CORS support that must be configured to allow cross-origin requests from frontend apps before security filters run.

@EnableWebSecurity — Annotation that enables Spring Security's web security support and allows customization via a SecurityFilterChain bean.

@EnableMethodSecurity — Annotation (replaced @EnableGlobalMethodSecurity) that enables method-level security annotations like @PreAuthorize and @PostAuthorize.

@PreAuthorize — A method-level annotation that evaluates a Spring Expression Language (SpEL) expression before the method is invoked to control access.

@PostAuthorize — A method-level annotation that evaluates a SpEL expression after the method returns, allowing access control based on the return value.

@Secured — A simpler method-level annotation that restricts access based on a list of role names (e.g., @Secured("ROLE_ADMIN")).

@RolesAllowed — A JSR-250 annotation equivalent to @Secured for restricting method access by role, supported via @EnableMethodSecurity.

Role — A coarse-grained permission label prefixed with ROLE_ (e.g., ROLE_ADMIN, ROLE_USER) assigned to a user to control broad access.

Permission (Authority) — A fine-grained privilege string (e.g., user:read, order:delete) assigned to a user for precise access control beyond roles.

hasRole() — A SpEL method in `@PreAuthorize` that checks whether the current user has a specific role (automatically adds `ROLE_` prefix).

hasAuthority() — A SpEL method in `@PreAuthorize` that checks whether the current user has a specific authority or permission string exactly as given.

hasAnyRole() — A SpEL method that returns true if the current user has at least one of the specified roles.

isAuthenticated() — A SpEL expression that returns true if the current user is authenticated (not anonymous).

isAnonymous() — A SpEL expression that returns true if the current user has not been authenticated.

permitAll() — A security rule that allows all users including unauthenticated ones to access the specified endpoint.

denyAll() — A security rule that blocks all users including authenticated ones from accessing the specified endpoint.

antMatchers() / **requestMatchers()** — Methods used in `HttpSecurity` to match URL patterns and apply authorization rules (antMatchers deprecated in Spring Security 6, replaced by requestMatchers).

Session Management — Spring Security's ability to control session creation strategy: ALWAYS, IF_REQUIRED, NEVER, or STATELESS for JWT-based APIs.

SessionCreationPolicy.STATELESS — Configures Spring Security to never create or use an HTTP session, required for stateless JWT-based REST APIs.

Remember Me — A Spring Security feature that persists authentication across browser sessions using a persistent token stored in a cookie and database.

Logout — Spring Security's built-in logout support that invalidates the session, clears the `SecurityContext`, and optionally deletes cookies on logout.

Basic Authentication — An HTTP authentication scheme where the username and password are base64-encoded and sent in the Authorization header with every request.

Form Login — Spring Security's built-in form-based login support that processes a POST to `/login` with username and password parameters.

ExceptionTranslationFilter — A Spring Security filter that handles `AuthenticationException` (triggers login) and `AccessDeniedException` (triggers 403 response).

AuthenticationEntryPoint — An interface invoked when an unauthenticated user attempts to access a protected resource; returns a 401 Unauthorized response.

AccessDeniedHandler — An interface invoked when an authenticated user lacks permission to access a resource; returns a 403 Forbidden response.

MethodSecurityExpressionHandler — The component that evaluates SpEL expressions in `@PreAuthorize` and `@PostAuthorize` annotations for method-level security.

InMemoryUserDetailsManager — A `UserDetailsService` implementation that stores user details in memory; suitable for testing and development only.

JdbcUserDetailsManager — A `UserDetailsService` implementation that loads user credentials and authorities from a relational database via JDBC.

SecurityMatcher — Restricts a `SecurityFilterChain` to apply only to requests matching a specific URL pattern, allowing multiple filter chains for different API areas.

Multiple Security Filter Chains — A Spring Security pattern where different filter chains (e.g., one for `/api/`, one for `/admin/`) apply different rules to different URL groups.

Content Security Policy (CSP) — An HTTP security header configurable in Spring Security that restricts which sources of content (scripts, styles) the browser can load.

HSTS (HTTP Strict Transport Security) — A security header Spring Security adds by default to force browsers to always use HTTPS for future requests to the domain.

X-Frame-Options — A security header Spring Security sets by default to prevent clickjacking by controlling whether the page can be embedded in an iframe.

X-Content-Type-Options — A security header that prevents browsers from MIME-sniffing a response away from the declared Content-Type, set by Spring Security by default.

Security Headers — A set of HTTP response headers (CSP, HSTS, X-Frame-Options, etc.) automatically added by Spring Security to protect against common web attacks.

IP-Based Access Control — Spring Security's ability to restrict endpoint access based on the client's IP address using `hasIpAddress()` in SpEL expressions.

Two-Factor Authentication (2FA) — An additional security layer requiring a second verification step (e.g., OTP) after password validation, implementable as a custom filter.

LDAP Authentication — Spring Security's support for authenticating users against an LDAP directory server, common in enterprise environments.

SAML — Security Assertion Markup Language; an XML-based SSO protocol supported via Spring Security SAML for enterprise identity federation.

SSO (Single Sign-On) — A session/authentication scheme that allows a user to log in once and access multiple applications without re-authenticating.

Delegating Password Encoder — Spring Security's `DelegatingPasswordEncoder` stores the encoding algorithm ID with the hash (e.g., `{bcrypt}$2a$...`) allowing multiple encoders simultaneously for migration.

Password Migration — Spring Security supports upgrading legacy password hashes (MD5, SHA) to BCrypt transparently at login time using DelegatingPasswordEncoder.

Method Security — The ability to secure individual service or repository methods using annotations (@PreAuthorize, @Secured) rather than only URL patterns.

Global Method Security — Application-wide activation of method-level security, enabled via @EnableMethodSecurity so annotations are enforced across all beans.

Anonymous Authentication — Spring Security creates an anonymous Authentication object for unauthenticated users so filters and expressions can handle them consistently.

RunAs Authentication — A Spring Security mechanism that temporarily replaces a user's Authentication with a different one during a method call for privilege escalation scenarios.

Security Event Publishing — Spring Security publishes events (AuthenticationSuccessEvent, AuthenticationFailureEvent, etc.) to the ApplicationContext for logging and auditing.

Audit Logging — Capturing and storing authentication and authorization events (login success/failure, access denied) for compliance and security monitoring.

Spring Data JPA

JPA — Java specification for ORM and database persistence.

@Entity — Marks class as JPA entity.

@Table — Maps entity to specific database table.

@Id — Marks primary key column.

@GeneratedValue — Automatically generates primary key values.

@Column — Maps field to database column.

@Transient — Marks a field to be excluded from persistence; it will not be mapped to any database column.

Repository — Interface layer for database operations.

CrudRepository — Provides basic CRUD methods: save(), findById(), findAll(), delete(), and count().

JpaRepository — Extends CrudRepository and PagingAndSortingRepository with JPA-specific methods and batch operations.

PagingAndSortingRepository — Extends CrudRepository with methods for paginated and sorted retrieval of entities.

save() — Persists a new entity (INSERT) or updates an existing one (UPDATE) based on the presence of an ID.

findById() — Returns an Optional containing the entity with the given primary key, or empty if not found.

findAll() — Retrieves all entities of the type from the database as a List.

deleteById() — Deletes the entity with the specified primary key from the database.

Derived Query Methods — Spring Data automatically generates SQL queries by parsing method names following a naming convention.

findByName() — Example of a derived query; Spring Data generates SELECT ... WHERE name = ? automatically.

@Query — Allows defining custom JPQL or native SQL queries directly on repository methods.

JPQL — Java Persistence Query Language; an object-oriented query language that operates on entity classes instead of tables.

Native Query — Executes raw SQL directly against the database using @Query(nativeQuery = true).

Pagination — Returns results as a Page object containing a subset of data along with total count and page metadata.

Pageable — Interface holding pagination parameters (page number, size, sort) passed to repository methods.

Sorting — Orders query results by one or more fields in ascending or descending direction using Sort.

Lazy Loading — Related entities are fetched from the database only when they are accessed for the first time.

Eager Loading — Related entities are fetched immediately together with the parent entity in a single query.

FetchType.LAZY — Configuration for loading a relationship only when explicitly accessed; default for @OneToMany and @ManyToMany.

FetchType.EAGER — Configuration for loading a relationship immediately with the parent; default for @ManyToOne and @OneToOne.

@OneToOne — Defines a one-to-one relationship between two entities mapped to two tables with a shared key.

@OneToMany — Defines a one-to-many relationship where one parent entity is associated with multiple child entities.

@ManyToOne — Defines a many-to-one relationship where multiple child entities point to a single parent entity.

@ManyToMany — Defines a many-to-many relationship between two entities, typically requiring a join table.

Cascade Type — Automatically propagates operations to child entities. Specifies which persistence operations (PERSIST, MERGE, REMOVE, etc.) should propagate from parent to child entity.

CascadeType.ALL — Propagates all JPA operations (PERSIST, MERGE, REMOVE, REFRESH, DETACH) from parent to child entities.

mappedBy — Attribute on the non-owning side of a bidirectional relationship pointing to the owning side's field name.

Join Column — The foreign key column in the database used to link two related entities.

@JoinColumn — Specifies the foreign key column name and other constraints for a relationship mapping.

N+1 Problem — Performance issue where fetching N parent entities triggers N additional queries to load their related entities.

Hibernate Session — The primary interface for interacting with the persistence context; equivalent to a JPA EntityManager.

Persistence context managing entity lifecycle.

Persistence Context — Cache storing managed entities temporarily. A first-level cache maintained by Hibernate that tracks managed entity instances within a session.

First Level Cache — The session-scoped cache that prevents duplicate queries for the same entity within a single transaction.(Session-level cache)

Second Level Cache — An optional, shared, application-level cache (e.g., EhCache) that persists across sessions for improved performance.(Session Factory Level cache)

Dirty Checking — Hibernate automatically detects changes to managed entities and generates UPDATE SQL at flush/commit time.

Transaction — A unit of work consisting of multiple DB operations that must all succeed or all be rolled back together.

@Transactional — Marks a method or class as transactional; Spring manages begin, commit, and rollback automatically.

ACID Properties — Database transaction guarantees: Atomicity, Consistency, Isolation, and Durability.

Flush — Synchronizes the in-memory persistence context state with the underlying database by executing pending SQL.

Commit — Permanently saves all changes made in the current transaction to the database.

Rollback — Undoes all changes made in the current transaction, restoring the database to its previous state.

Optimistic Locking — A concurrency strategy that uses a version field to detect and reject concurrent conflicting updates.

@Version — Enables optimistic locking. Marks a field as the version number for optimistic locking; Hibernate increments it on every update.

Pessimistic Locking — Locks database rows during transaction execution. Acquires a database-level lock on a row during a transaction, preventing concurrent modifications.

Projection — Fetches selected columns instead of full entity. A technique to retrieve only specific columns rather than full entity objects, improving query performance.

DTO Projection — Maps query result directly into DTO object. Maps query results directly into a DTO/interface, avoiding loading entire entity objects unnecessarily.

Specification API — Builds dynamic queries programmatically.

Criteria API — Type-safe dynamic query generation.

Auditing — Automatically tracks created/updated details.

@CreatedDate — Stores entity creation timestamp.

@LastModifiedDate — Stores entity update timestamp.

EntityManager — Core JPA interface for persistence operations.

Hibernate Dialect — Adapts Hibernate SQL generation for specific DB.

Open Session in View — Keeps Hibernate session open during request lifecycle.

Java Commonly Asked Questions Answers

JDK — Java Development Kit containing tools for Java development.

JRE — Java Runtime Environment used to run Java applications.

JVM — Java Virtual Machine executing Java bytecode.

Bytecode — Platform-independent code generated by Java compiler.

Platform Independence — Java code runs on any OS using JVM.

WORA — “Write Once Run Anywhere” principle of Java.

OOPS — Programming paradigm based on objects and classes.

Class — Blueprint/template for creating objects.

Object — Instance of a class containing state and behavior.

Encapsulation — Wrapping data and methods together.

Abstraction — Hiding implementation details and exposing essentials only.

Inheritance — Acquiring properties and behaviors from parent class.

Polymorphism — Same method behaving differently in different contexts.

Method Overloading — Same method name with different parameters.

Method Overriding — Child class redefining parent class method.

Constructor — Special method initializing objects.

Default Constructor — Constructor automatically added by compiler if absent.

Parameterized Constructor — Constructor accepting arguments.

this Keyword — Refers to current object instance.

super Keyword — Refers to parent class members.

Static Keyword — Belongs to class rather than object.

final Keyword — Prevents modification, inheritance, or overriding.

finally Block — Executes regardless of exception occurrence.

finalize() — Old GC cleanup method called before object destruction.

Access Modifier — Controls visibility/accessibility of classes and members.

public — Accessible everywhere.

private — Accessible only within same class.

protected — Accessible within package and subclasses.

default Access — Accessible only within same package.

Interface — Blueprint containing abstract methods and constants.

Abstract Class — Incomplete class containing abstract/concrete methods.

Marker Interface — Empty interface providing metadata to JVM/frameworks.

Serializable — Marker interface enabling object serialization.

Serialization — Converting object into byte stream.

Deserialization — Converting byte stream back into object.

transient Keyword — Prevents field serialization.

volatile Keyword — Ensures variable visibility across threads.

enum — Special datatype representing fixed constants.

Package — Namespace organizing related classes/interfaces.

Import Statement — Allows usage of external packages/classes.

Exception — Event disrupting normal program flow.

Checked Exception — Compiler-enforced exception handling.

Unchecked Exception — Runtime exceptions not checked at compile time.

try-catch — Handles exceptions gracefully.

throw — Explicitly throws exception manually.

throws — Declares possible exceptions from method.

Custom Exception — User-defined exception class.

Memory Leak — Unused objects occupying memory unnecessarily.

Garbage Collection — Automatic cleanup of unused objects.

Heap Memory — Stores objects and instance variables.

Stack Memory — Stores method calls and local variables.

String — Immutable sequence of characters.

String Pool — Special heap area storing reusable string literals.

StringBuilder — Mutable string class optimized for performance.

StringBuffer — Thread-safe mutable string class.

Immutable Object — Object whose state cannot change after creation.

== Operator — Compares object references.

equals() — Compares object contents logically.

hashCode() — Generates integer hash representation of object.

Comparable — Interface for natural sorting using compareTo().

Comparator — Interface for custom sorting logic.

Collection Framework — Set of classes/interfaces for data handling.

List — Ordered collection allowing duplicates.

Set — Collection storing unique elements only.

Map — Key-value pair collection.

ArrayList — Dynamic array implementation with fast random access.

LinkedList — Doubly linked list implementation optimized for insertion/deletion.

Vector — Thread-safe dynamic array.

HashSet — Unordered collection storing unique elements.

TreeSet — Sorted set implementation using tree structure.

HashMap — Key-value collection using hashing.

LinkedHashMap — Maintains insertion order in map.

TreeMap — Sorted map implementation using Red-Black Tree.

Hashtable — Synchronized thread-safe HashMap.

ConcurrentHashMap — High-performance thread-safe HashMap.

Iterator — Used for traversing collections sequentially.

Fail-Fast Iterator — Throws exception if collection modified during iteration.

Fail-Safe Iterator — Works on collection copy avoiding exceptions.

Multithreading — Executing multiple threads concurrently.

Thread — Smallest independent execution unit.

Runnable — Interface defining task for thread execution.

Synchronization — Controlling shared resource access between threads.

Deadlock — Threads waiting indefinitely for each other's locks.

ExecutorService — Framework managing thread pools efficiently.

Lambda Expression — Concise anonymous function introduced in Java 8.

Functional Interface — Interface containing single abstract method.

Stream API — Functional-style collection processing framework.

Optional — Container preventing NullPointerException.

Reflection API — Inspects/modifies classes and methods dynamically.

Annotation — Metadata added to Java code.

JDBC — API for Java database connectivity.

Connection Pooling — Reusing database connections efficiently.
JIT Compiler — Converts bytecode into optimized machine code.
ClassLoader — Dynamically loads Java classes into JVM memory.
Singleton Pattern — Ensures only one object instance exists.
Factory Pattern — Creates objects without exposing creation logic.
Dependency Injection — Dependencies provided externally to objects.
SOLID Principles — Five principles for maintainable object-oriented design.
S — Single Responsibility Principle — A class should have only one reason to change.
O — Open Closed Principle — Open for extension but closed for modification.
L — Liskov Substitution Principle — Child classes should replace parent classes without breaking behavior.
I — Interface Segregation Principle — Clients should not depend on unused methods.
D — Dependency Inversion Principle — Depend on abstractions, not concrete implementations.

Multithreading

Multithreading — Executing multiple threads concurrently within a single process to improve performance and responsiveness.
Thread — Smallest independent execution unit inside a process.
Process — An independent program in execution with its own dedicated memory space, resources, and at least one thread.
Concurrency — The ability to manage and make progress on multiple tasks at overlapping time periods, not necessarily simultaneously.
Parallelism — Literally executing multiple tasks at the exact same time on multiple CPU cores.
Main Thread — The default thread created when a Java program starts, from which other threads are spawned.
Thread Lifecycle — The sequence of states a thread passes through: New → Runnable → Running → Blocked/Waiting → Terminated.
New State — A thread that has been created using new Thread() but the start() method has not yet been called.
Runnable State — A thread that is eligible to run and waiting to be selected by the CPU scheduler, or currently running.
Blocked State — A thread that is waiting to acquire an object's monitor lock held by another thread.
Waiting State — A thread that is indefinitely waiting for another thread to perform a specific action (e.g., notify()).
Timed Waiting — A thread that waits for a specified time duration using sleep(), join(timeout), or wait(timeout).
Terminated State — A thread that has completed execution or was stopped due to an exception.
Thread Class — Java class used to create threads by extending it and overriding the run() method.
Runnable Interface — Preferred functional interface for defining thread tasks; implemented and passed to a Thread constructor.
Callable Interface — Similar to Runnable but the call() method can return a result and throw checked exceptions.
Future — An object that holds the result of an asynchronous Callable task and allows retrieving it once computation is complete.
Executor Framework — Higher-level API in java.util.concurrent that manages thread creation, pooling, and task submission.
Thread Pool — A collection of pre-created worker threads that are reused for executing submitted tasks efficiently.
Executors Class — Factory class providing static methods to create various types of thread pools.
Fixed Thread Pool — A thread pool with a fixed number of threads; tasks wait in a queue if all threads are busy.
Cached Thread Pool — A thread pool that creates new threads as needed and reuses idle ones; suitable for short-lived tasks.
Single Thread Executor — Executes submitted tasks sequentially one at a time using a single background thread.
Scheduled Executor Service — Executes tasks after a fixed delay or at recurring intervals, similar to a cron job.
start() — Initiates a new thread of execution, invoking the run() method in a separate thread.
run() — Contains the logic to be executed by the thread; calling run() directly does NOT start a new thread.
sleep() — Pauses the current thread's execution for a specified millisecond duration without releasing locks.
join() — Causes the calling thread to wait until the thread it is called on finishes execution.
yield() — Hints to the scheduler that the current thread is willing to yield its CPU time slice to other threads.
Daemon Thread — A low-priority background thread (e.g., garbage collector) that terminates automatically when all user threads finish.
User Thread — A regular foreground thread; the JVM continues running until all user threads have completed.
Synchronization — Mechanism to ensure only one thread at a time can access a shared resource or critical section.
Synchronized Method — Acquires the object's intrinsic lock for the entire method, preventing concurrent access.
Synchronized Block — Acquires the lock only for a specific block of code, offering finer-grained control than a synchronized method.
Monitor Lock — The intrinsic lock (mutex) associated with every Java object, used by the synchronized keyword.
Race Condition — A bug where multiple threads access and modify shared data concurrently, producing unpredictable results.
Critical Section — A portion of code that accesses shared resources and must not be executed by more than one thread at a time.
Thread Safety — A property of code that behaves correctly and produces consistent results when accessed by multiple threads simultaneously.
Deadlock — A situation where two or more threads are each waiting for a lock held by another, causing permanent blocking.
Starvation — A condition where a thread is perpetually denied CPU access because higher-priority threads always run first.

Livelock — A situation where threads keep responding to each other's actions but make no actual progress.

wait() — Causes the current thread to release the monitor lock and wait until another thread calls `notify()` or `notifyAll()`.

notify() — Wakes up a single thread that is waiting on the same object's monitor.

notifyAll() — Wakes up all threads waiting on the same object's monitor; they compete to re-acquire the lock.

volatile Keyword — Ensures that reads/writes to a variable are directly from/to main memory, guaranteeing visibility across threads.

Atomic Variable — A variable that supports lock-free, thread-safe operations using CPU-level compare-and-swap instructions.

AtomicInteger — Thread-safe integer class in `java.util.concurrent.atomic` supporting atomic increment, decrement, and CAS operations.

ReentrantLock — An explicit lock in `java.util.concurrent.locks` with features like fairness, `tryLock()`, and interruptible locking.

ReadWriteLock — A lock that allows multiple concurrent reads but exclusive access for writes, improving read-heavy performance.

Semaphore — A concurrency utility that controls the number of threads that can access a resource simultaneously.

CountDownLatch — A synchronization barrier that makes one or more threads wait until a set of operations in other threads complete.

CyclicBarrier — A synchronization aid that makes a group of threads wait for each other to reach a common barrier point.

ForkJoinPool — A specialized thread pool for divide-and-conquer parallelism using the work-stealing algorithm.

CompletableFuture — A powerful class for writing non-blocking asynchronous code with chaining, combining, and exception handling.

ConcurrentHashMap — Thread-safe, high-performance map that allows concurrent reads and fine-grained segment-level locking for writes.

CopyOnWriteArrayList — Thread-safe list that creates a fresh copy of the underlying array on every write, ideal for read-heavy workloads.

BlockingQueue — A thread-safe queue that blocks producers when full and blocks consumers when empty, used in producer-consumer patterns.

Producer-Consumer Problem — Classic concurrency problem where producers add items to a shared buffer and consumers remove them, requiring synchronization.

Context Switching — The OS process of saving a thread's state and loading another's, allowing multiple threads to share CPU time.

Thread Priority — An integer hint (1–10 in Java) that suggests to the OS scheduler the relative importance of a thread.

Immutable Object — An object whose state cannot be modified after creation, making it inherently thread-safe.

ThreadLocal — Provides each thread with its own independent copy of a variable, avoiding shared state between threads.

Parallel Stream — Java Stream that uses the `ForkJoinPool` to process elements concurrently across multiple CPU cores.

Fork-Join Framework — A parallel execution framework that recursively splits a task into subtasks, executes them in parallel, and joins results.

CAS (Compare And Swap) — A lock-free atomic operation that updates a variable only if it currently holds an expected value.

Busy Waiting — An antipattern where a thread repeatedly checks a condition in a loop, wasting CPU cycles.

Lock Contention — A performance bottleneck where multiple threads frequently compete to acquire the same lock.

Visibility Problem — A concurrency issue where one thread's update to a shared variable is not seen by another thread due to CPU caching.

Memory Leak in Threads — Occurs when threads hold references to objects that are never released, preventing garbage collection.

CPU Bound Task — A task that spends most of its time doing computation, bottlenecked by CPU speed.

IO Bound Task — A task that spends most of its time waiting for I/O operations like network or disk access.

Unit Testing

Unit Testing — Testing the smallest testable parts of an application (methods, classes) in isolation from external dependencies.

JUnit — The most widely used Java testing framework providing annotations, assertions, and test lifecycle management.

JUnit 5 — The latest major version of JUnit with a modular architecture (Jupiter, Vintage, Platform) and improved extensibility.

Test Case — A single test method annotated with `@Test` that verifies one specific behavior or scenario.

Test Suite — A collection of test classes/methods grouped together and executed as a single run.

Assertion — A statement that checks whether the actual result matches the expected result; fails the test if not.

assertEquals() — Asserts that two values are equal; fails the test if they differ.

assertNotEquals() — Asserts that two values are not equal; fails the test if they are the same.

assertTrue() — Asserts that the given boolean condition evaluates to true.

assertFalse() — Asserts that the given boolean condition evaluates to false.

assertNull() — Asserts that the given object reference is null.

assertNotNull() — Asserts that the given object reference is not null.

assertThrows() — Asserts that executing a given block of code throws a specific exception type.

@Test — Marks a method as a JUnit test case that should be discovered and executed by the test runner.

@BeforeEach — Annotates a setup method that runs before every individual test method in the class.

@AfterEach — Annotates a teardown method that runs after every individual test method in the class.

@BeforeAll — Annotates a static setup method that runs once before all test methods in the class.

@AfterAll — Annotates a static teardown method that runs once after all test methods in the class.

@Disabled — Disables a specific test class or method from being executed during the test run.

@DisplayName — Provides a human-readable custom name for a test class or method, shown in test reports.

Parameterized Test — A test that runs multiple times with different input values to verify behavior across many cases.

@ParameterizedTest — JUnit 5 annotation that marks a test to run with multiple arguments from a source (e.g., @ValueSource).

Test Coverage — The percentage of production code lines, branches, or methods exercised by the test suite.

Code Coverage Tool — Tool that instruments bytecode to measure which code paths are executed during testing.

JaCoCo — Java Code Coverage library that generates line/branch coverage reports and integrates with Maven/Gradle.

Mocking — Creating a controlled, fake replacement for a real dependency to test a class in complete isolation.

Mockito — The most popular Java mocking framework for creating mock objects, stubbing behavior, and verifying interactions.

Mock Object — A fake object that simulates the behavior of a real dependency with predefined stubbed responses.

@Mock — Mockito annotation that creates and injects a mock object into the test class.

@InjectMocks — Mockito annotation that creates the class under test and injects all @Mock/@Spy fields into it.

@Spy — Mockito annotation that wraps a real object, allowing partial mocking of only specific methods.

when().thenReturn() — Stubs a mock method to return a specific value when called with given arguments.

verify() — Asserts that a specific method was called on a mock object with expected arguments and invocation count.

times() — Used with verify() to assert a method was invoked exactly a specified number of times.

any() — An argument matcher that matches any value of the expected type in Mockito stubbing or verification.

doThrow() — Stubs a void mock method to throw a specified exception when invoked.

doNothing() — Stubs a void mock method to do nothing when invoked (default behavior for void mocks).

Stub — A test double that provides canned, predefined answers to method calls made during the test.

Fake Object — A simpler, working implementation of a dependency used in tests (e.g., an in-memory database).

Integration Testing — Tests that verify the interaction and collaboration between multiple real components or layers.

Spring Boot Test — Spring Boot's testing support that provides annotations and utilities for integration and slice testing.

@SpringBootTest — Loads the full Spring ApplicationContext for end-to-end integration testing of a Spring Boot application.

@WebMvcTest — Loads only the Spring MVC layer (controllers, filters) without starting the full application context.

@DataJpaTest — Loads only the JPA layer with an in-memory database for testing repositories in isolation.

MockMvc — Spring MVC test framework that allows performing and asserting HTTP requests without starting a real server.

TestContainer — A Java library that runs real Docker containers (databases, services) during tests for realistic integration testing.

H2 Database — An in-memory relational database commonly used as a lightweight substitute for real databases during testing.

BDD Testing — Behavior-Driven Development testing style that describes tests in terms of user-facing behavior and scenarios.

Given-When-Then — BDD structure: Given (precondition), When (action), Then (expected outcome); improves test readability.

TDD — Test-Driven Development; writing failing tests first, then writing the minimal code to pass them, then refactoring.

AAA Pattern — Arrange (set up data), Act (call the method), Assert (verify the result); standard unit test structure.

Regression Testing — Re-running existing tests after code changes to ensure previously working functionality still works.

Smoke Testing — A quick, high-level test pass that verifies the most critical features of an application work after a build.

End-to-End Testing — Tests the complete user journey through the full stack, from UI to database and back.

Test Fixture — The set of preconditions, test data, and environment configuration needed to execute a test.

Dependency Injection Testing — Injecting mock or fake dependencies into the class under test to achieve full isolation.

Isolation in Unit Testing — Ensuring the unit being tested does not interact with real databases, files, or network calls.

Continuous Testing — Running automated tests automatically on every code commit as part of a CI/CD pipeline.

SonarQube — Static code analysis platform that reports on code quality, bugs, vulnerabilities, and test coverage.

Mutation Testing — Technique that introduces small intentional bugs (mutations) into production code to verify that tests catch them.

Thread Safety Testing — Verifying that shared code or data structures behave correctly under concurrent multi-threaded access.

Performance Testing — Evaluates system responsiveness, throughput, and stability under expected and peak load conditions.

Load Testing — Tests how a system behaves when subjected to a high volume of concurrent users or requests.

Test Pyramid — Testing strategy advocating for a large base of fast unit tests, fewer integration tests, and even fewer UI tests.

SQL

Table — Collection of rows and columns storing data.

Row — Single record in a table.

Column — Attribute/field in a table.

Primary Key — Unique identifier for each table record.

Foreign Key — Column creating relationship between tables.

Composite Key — Primary key consisting of multiple columns.

Candidate Key — Column eligible to become primary key.

Unique Key — Ensures column values are unique.

Super Key — Set of columns uniquely identifying records.

Constraint — Rule enforcing data integrity in database.

NOT NULL — Prevents column from storing null values.

Default Constraint — Provides default value if none supplied.

Check Constraint — Restricts column values based on condition.

Index — Data structure improving query performance.

Clustered Index — Physically sorts table data based on index key.

Non-Clustered Index — Separate structure storing indexed column references.

Composite Index — Index created on multiple columns.

Covering Index — Index containing all columns needed by query.

Normalization — Organizing data to reduce redundancy.

Denormalization — Adding redundancy to improve performance.

1NF — Eliminates repeating groups and ensures atomic columns.

2NF — Removes partial dependency from composite keys.

3NF — Removes transitive dependency between columns.

BCNF — Stronger version of 3NF handling candidate key anomalies.

SELECT — Retrieves data from database tables.

WHERE — Filters rows based on condition.

GROUP BY — Groups rows sharing same values.

HAVING — Filters grouped records after aggregation.

ORDER BY — Sorts query results ascending or descending.

DISTINCT — Removes duplicate rows from result set.

LIMIT — Restricts number of returned rows.

OFFSET — Skips specified number of rows.

JOIN — Combines rows from multiple tables.

INNER JOIN — Returns matching rows from both tables.

LEFT JOIN — Returns all left table rows and matching right rows.

RIGHT JOIN — Returns all right table rows and matching left rows.

FULL OUTER JOIN — Returns matching and non-matching rows from both tables.

CROSS JOIN — Produces Cartesian product of tables.

SELF JOIN — Table joined with itself.

UNION — Combines results removing duplicates.

UNION ALL — Combines results including duplicates.

Subquery — Query nested inside another query.

Correlated Subquery — Subquery dependent on outer query execution.

View — Virtual table based on query result.

Materialized View — Physically stored query result for faster access.

Stored Procedure — Precompiled SQL block reusable multiple times.

Function — Reusable SQL logic returning value.

Trigger — Automatically executes on table events like insert/update/delete.

Cursor — Processes query result row by row.

Transaction — Group of SQL operations executed atomically.

Commit — Permanently saves transaction changes.

Rollback — Reverts transaction changes.

Savepoint — Partial rollback point inside transaction.

ACID Properties — Atomicity, Consistency, Isolation, Durability in transactions.

Atomicity — Transaction fully succeeds or fully fails.

Consistency — Database remains valid before and after transaction.

Isolation — Transactions execute independently without interference.

Durability — Committed data persists permanently.

Deadlock — Transactions waiting indefinitely for each other's locks.

Locking — Restricting simultaneous data access for consistency.

Shared Lock — Allows read-only access to data.

Exclusive Lock — Prevents others from reading/writing data.

Optimistic Locking — Assumes conflicts are rare and validates before commit.

Pessimistic Locking — Locks data immediately to prevent conflicts.

Dirty Read — Reading uncommitted transaction data.

Non-Repeatable Read — Same query returning different results within transaction.

Phantom Read — New rows appearing during same transaction execution.

Isolation Level — Defines transaction visibility rules.

Read Uncommitted — Lowest isolation allowing dirty reads.

Read Committed — Prevents dirty reads only.
Repeatable Read — Prevents non-repeatable reads.
Serializable — Highest isolation preventing phantom reads.
Execution Plan — Database strategy used to execute query.
EXPLAIN — Displays query execution plan.
Full Table Scan — Database scans entire table due to missing index.
Cardinality — Number of unique values in column.
Partitioning — Splitting large tables into smaller manageable parts.
Sharding — Distributing database across multiple servers.
Replication — Copying database data across servers.
OLTP — Transaction-oriented operational database systems.
OLAP — Analytical systems optimized for reporting and queries.
CTE — Temporary named result set inside query.
Window Function — Performs calculations across related row sets.
ROW_NUMBER() — Assigns sequential row numbers.
RANK() — Assigns ranking with gaps for ties.
DENSE_RANK() — Assigns ranking without gaps.
LEAD() — Accesses next row value.
LAG() — Accesses previous row value.
Database Migration — Managing schema changes across environments.

Git

Git — A free, open-source distributed version control system for tracking changes in source code and coordinating work among developers.

Repository (Repo) — A directory tracked by Git containing all project files, commit history, branches, and configuration.

Local Repository — A Git repository stored on a developer's local machine, including the full history and all branches.

Remote Repository — A Git repository hosted on a server or platform (e.g., GitHub, GitLab) shared among team members.

GitHub — A cloud-based platform for hosting Git repositories with features for collaboration, pull requests, and CI/CD.

GitLab — A DevOps platform offering Git repository hosting alongside built-in CI/CD, issue tracking, and security scanning.

Bitbucket — Atlassian's Git repository hosting service that integrates tightly with Jira and other Atlassian tools.

Version Control — A system for recording and managing changes to files over time, enabling history, collaboration, and rollback.

Commit — A snapshot of the staged changes saved permanently into the repository's history with a message and timestamp.

Commit ID (Hash) — A unique 40-character SHA-1 hash that uniquely identifies each commit in the repository history.

Branch — A lightweight, movable pointer to a specific commit, allowing isolated parallel lines of development.

Main/Master Branch — The default primary branch that typically holds stable, production-ready code.

Feature Branch — A short-lived branch created from main to develop a specific feature before merging back.

Checkout — Switches the working directory and HEAD pointer to a different branch or specific commit.

Clone — Creates a complete local copy of a remote repository including all branches, commits, and history.

Fork — Creates a personal copy of someone else's repository on a hosting platform, allowing independent modifications.

Pull — Fetches changes from the remote repository and merges them into the current local branch.

Push — Uploads local commits from the current branch to the corresponding branch on the remote repository.

Fetch — Downloads the latest changes from the remote repository without merging them into the local working branch.

Merge — Integrates the changes from one branch into another, creating a merge commit if histories diverge.

Merge Conflict — Occurs when two branches have modified the same lines of a file differently and Git cannot auto-resolve them.

Rebase — Moves or replays commits from one branch on top of another branch for a linear, cleaner history.

Cherry Pick — Applies the changes from a specific commit onto the current branch without merging the entire branch.

Stash — Temporarily shelves uncommitted changes (tracked and staged) so you can switch context without committing.

Pop Stash — Restores the most recent stashed changes back to the working directory and removes the stash entry.

Stage Area (Index) — The intermediate area where changes are gathered with git add before they are committed.

git add — Moves specified changes from the working directory into the staging area for inclusion in the next commit.

git commit — Records all staged changes into the repository history with a descriptive commit message.

git status — Displays the current state of the working directory and staging area, showing modified, staged, and untracked files.

git log — Displays the commit history of the current branch including commit hash, author, date, and message.

git diff — Shows the line-by-line differences between the working directory, staging area, or commits.

HEAD — A special Git pointer that refers to the currently checked-out commit or the tip of the active branch.

Detached HEAD — A state where HEAD points directly to a commit hash instead of a branch name; commits here can be lost.

Tag — A named, immovable reference to a specific commit, typically used to mark release versions.

Annotated Tag — A tag stored as a full Git object containing tagger name, date, message, and optionally a signature.

Lightweight Tag — A simple pointer directly to a commit without any additional metadata or object storage.

Pull Request (PR) — A GitHub feature for proposing changes from one branch to another and requesting code review before merging.

Merge Request (MR) — GitLab's equivalent of a GitHub Pull Request for proposing and reviewing branch merges.

Code Review — The practice of examining proposed code changes before merging to catch bugs and maintain quality.

Conflict Resolution — The manual process of editing conflicting code sections and marking them as resolved before committing.

git reset — Moves HEAD and optionally the staging area or working directory to a previous commit state.

Soft Reset — Moves HEAD to a specified commit while keeping all changes staged in the index.

Mixed Reset — Moves HEAD to a specified commit and unstages changes, but keeps them in the working directory (default).

Hard Reset — Moves HEAD to a specified commit and permanently discards all staged and unstaged changes.

git revert — Creates a new commit that undoes the changes from a specific previous commit, preserving history.

git rm — Removes a tracked file from both the working directory and the Git index.

git mv — Renames or moves a tracked file and stages the rename, equivalent to `mv + git rm + git add`.

git ignore — The concept of telling Git to exclude specific files or patterns from tracking using a `.gitignore` file.

.gitignore — A file listing file name patterns Git should ignore; commonly used for build artifacts and secrets.

Upstream Branch — The remote-tracking branch that a local branch is set to track and sync with by default.

Origin — The conventional default name for the primary remote repository a local repository was cloned from.

Fast Forward Merge — A merge where Git simply moves the branch pointer forward without creating a merge commit.

Squash Merge — Combines all commits from a feature branch into a single commit before merging into the target branch.

Bisect — A Git command that performs a binary search through commit history to find the commit that introduced a bug.

Hook — Custom scripts placed in `.git/hooks` that Git executes automatically at specific points like commit or push.

Pre-Commit Hook — A hook that runs before a commit is created, often used to enforce linting or run tests.

CI/CD Integration — Automating build, test, and deployment workflows triggered by Git push or merge events.

Git Workflow — A defined branching strategy and collaboration process for managing code changes in a team.

Git Flow — A branching model using `main`, `develop`, `feature/`, `release/`, and `hotfix/` branches for structured releases.

Trunk-Based Development — A branching strategy where all developers commit directly to the main branch frequently, keeping branches short-lived.

Shallow Clone — Cloning a repository with a limited commit history depth (`--depth N`) for faster downloads.

Submodule — A Git repository embedded inside another repository, tracked at a specific commit.

Blame — A Git command showing the author, commit, and timestamp of every line in a file.

git reflog — Records every movement of HEAD, allowing recovery of lost commits or branches that are no longer referenced.

Detached Changes Recovery — Using `git reflog` to find and restore commits that were made in detached HEAD state.

Working Directory — The local directory containing the actual project files currently being edited.

Index — Another name for the Git staging area; the intermediate snapshot prepared for the next commit.

Remote Tracking Branch — A read-only local reference (e.g., `origin/main`) that mirrors the state of a branch on the remote.

Bare Repository — A Git repository without a working directory, used as a central shared repository (e.g., on a server).

Monorepo — A single Git repository that contains the source code of multiple related projects or services.

Patch — A file representing a set of code changes as a diff, used for reviewing or sharing changes outside of Git pushes.

Git Archive — Exports the files of a repository at a given commit or tag into a tar/zip archive, without including `.git` history.

CI/CD Jenkins One-Liners

CI/CD — Automates code integration, testing, and deployment.

CI (Continuous Integration) — Frequently merging code changes with automated build and testing.

CD (Continuous Delivery) — Automatically prepares code for production deployment.

CD (Continuous Deployment) — Automatically deploys every successful change to production.

Pipeline — Sequence of automated build, test, and deployment stages.

Build — Process of compiling and packaging application code.

Artifact — Deployable output generated from a build.

Source Control — System for tracking code changes (e.g., Git).

Commit — Snapshot of code changes stored in repository.

Branch — Independent line of development.

Pull Request (PR) — Request to review and merge code changes.

Code Review — Validation of code quality before merging.

Automated Testing — Running tests automatically during pipeline execution.

Unit Test — Tests individual methods/classes in isolation.

Integration Test — Tests interaction between multiple components.

Regression Test — Ensures new changes don't break existing functionality.

Deployment — Releasing application to target environment.

Rollback — Reverting application to previous stable version.

Blue-Green Deployment — Switches traffic between old and new environments.

Canary Deployment — Releases new version to a small percentage of users first.

Rolling Deployment — Gradually replaces old instances with new ones.

Infrastructure as Code (IaC) — Managing infrastructure through code instead of manual setup.

Containerization — Packaging application and dependencies together.

Docker — Platform for creating and running containers.

Kubernetes — Platform for deploying and managing containers at scale.

Jenkins — Popular CI/CD automation server.

GitHub Actions — CI/CD automation built into GitHub.

GitLab CI/CD — Built-in CI/CD platform in GitLab.

Azure DevOps — Microsoft platform for CI/CD and project management.

Monitoring — Tracking application health after deployment.

Alerting — Notifying teams when failures or issues occur.

DevOps — Culture combining development and operations for faster delivery.

CI — Automatically build and test code after every change.

CD — Automatically deliver/deploy tested code.

Pipeline — Automated workflow from code commit to deployment.

Artifact — Deployable package generated from build.

Blue-Green — Two environments; switch traffic instantly.

Canary — Deploy to a small user group first.

Rollback — Revert to previous stable version.

Java + Spring Boot + Microservices Production Issues (One-Liner Answers)

Application Issues

- **Application not starting** — Check startup logs for bean/configuration errors.
- **Port already in use** — Identify process and free/change port.
- **High application startup time** — Analyze bean initialization and external calls.
- **Application crashing frequently** — Check logs, heap dumps, and resource usage.
- **OutOfMemoryError** — Increase heap or fix memory leak.
- **StackOverflowError** — Fix infinite recursion or circular calls.
- **High CPU usage** — Analyze thread dumps and CPU-intensive code.
- **High memory usage** — Check object retention and heap dump analysis.

Database Issues

- **Database connection exhausted** — Increase pool size or fix connection leaks.
- **Slow query** — Analyze execution plan and add proper indexes.
- **Deadlock detected** — Reduce lock scope and optimize transaction order.
- **Database CPU high** — Tune queries and indexing strategy.
- **Connection timeout** — Verify DB availability and connection pool settings.
- **N+1 Query Problem** — Use fetch joins or batch fetching.
- **Too many DB calls** — Introduce caching and batch processing.
- **Table lock issue** — Reduce transaction duration and optimize queries.

Microservices Issues

- **Service-to-service call failure** — Check network, DNS, and service health.
- **Service unavailable (503)** — Verify pods/instances are healthy.
- **Increased latency between services** — Analyze network and downstream dependencies.
- **Cascading failure** — Implement circuit breaker pattern.
- **Service discovery failure** — Verify registry and service registration.
- **Version compatibility issue** — Use backward-compatible APIs.
- **Distributed transaction failure** — Implement Saga pattern.
- **Duplicate request processing** — Use idempotency keys.

REST API Issues

- **API response slow** — Check DB queries, external calls, and caching.
- **Frequent 500 errors** — Analyze application logs and exception traces.
- **Frequent 404 errors** — Verify API routes and gateway configuration.
- **Request timeout** — Optimize processing or increase timeout.

- **Payload too large** — Compress data or paginate responses.
- **API rate limit exceeded** — Implement throttling and rate limiting.

Kafka Issues

- **Consumer lag increasing** — Scale consumers or optimize processing.
- **Messages not consumed** — Verify consumer group and offsets.
- **Messages duplicated** — Implement idempotent processing.
- **Messages lost** — Verify ACK settings and replication factor.
- **Consumer rebalance frequently** — Tune session timeout and heartbeat.
- **Partition skew** — Improve partition key selection.
- **Producer timeout** — Check broker health and network latency.
- **Broker down** — Verify replication and failover setup.

Cache Issues

- **Cache not updating** — Verify cache eviction strategy.
- **Cache stale data** — Configure proper TTL.
- **Low cache hit ratio** — Cache frequently accessed data.
- **Redis memory full** — Increase memory or configure eviction policy.
- **Cache stampede** — Use distributed locking or cache warming.

Kubernetes Issues

- **Pod CrashLoopBackOff** — Check application logs and startup failures.
- **Pod OOMKilled** — Increase memory limits or optimize memory usage.
- **Pod pending** — Verify node resources and scheduling constraints.
- **Service not reachable** — Check service selector and endpoints.
- **Ingress not working** — Verify ingress rules and controller health.
- **Too many pod restarts** — Check health checks and application stability.
- **Node pressure issue** — Scale cluster or optimize resources.

AWS Issues

- **EC2 CPU high** — Scale instance or optimize application.
- **RDS slow performance** — Analyze queries and increase resources.
- **S3 access denied** — Verify IAM roles and bucket policies.
- **Lambda timeout** — Optimize execution or increase timeout.
- **SQS backlog growing** — Scale consumers.
- **SNS notification failure** — Verify subscriptions and permissions.
- **MSK broker issue** — Check broker health and replication.

Security Issues

- **JWT token invalid** — Verify secret key and expiration.
- **Authentication failure** — Check credentials and identity provider.
- **Authorization failure (403)** — Verify roles and permissions.
- **Secrets exposed** — Move secrets to Secret Manager/Vault.
- **SSL certificate expired** — Renew certificate before expiration.

CI/CD Issues

- **Build failure** — Check compilation and dependency errors.
- **Deployment failure** — Verify configuration and environment variables.
- **Pipeline stuck** — Check runner/agent health.
- **Rollback required** — Revert to last stable release.
- **Configuration mismatch** — Externalize configs and validate environments.

Monitoring Issues

- **Application down** — Check health endpoint and logs.
- **No logs visible** — Verify logging configuration and collectors.
- **Alerts not triggered** — Validate alert rules and thresholds.
- **High error rate** — Analyze exceptions and failing endpoints.
- **Slow response time** — Check DB, cache, and external dependencies.

Top 15 Most Asked Production Issues

1. OutOfMemoryError → Analyze heap dump.
2. High CPU → Analyze thread dump.
3. Slow API → Check DB queries and external calls.
4. DB Connection Pool Exhausted → Fix connection leak.
5. Deadlock → Optimize transaction order.
6. Consumer Lag → Scale Kafka consumers.
7. Kafka Duplicate Messages → Make consumer idempotent.
8. Service Timeout → Circuit breaker + retry.
9. Cache Stale Data → Proper TTL strategy.
10. Pod CrashLoopBackOff → Check startup logs.
11. Pod OOMKilled → Increase memory or optimize code.
12. 503 Service Unavailable → Verify pod health and load balancing.
13. Deployment Failure → Verify configs and secrets.
14. SSL Certificate Expired → Renew certificates proactively.
15. Cascading Failure → Circuit breaker + bulkhead pattern.

Threading Issues

- **Thread Pool Exhausted** → Increase pool size or fix long-running tasks.
- **Thread Deadlock** → Analyze thread dump and fix lock ordering.
- **Too Many Threads** → Use thread pools instead of creating threads manually.
- **Blocked Threads** → Identify slow DB/API calls holding threads.

JVM Issues

- **Frequent Full GC** → Reduce object creation and tune heap settings.
- **Memory Leak** → Analyze heap dump using MAT/VisualVM.
- **GC Pause Time High** → Tune GC or upgrade to G1/ZGC.
- **ClassLoader Leak** → Release references during redeployments.

API / Integration Issues

- **Third-Party API Slow** → Use timeout, retry, and circuit breaker.
- **External API Down** → Fallback mechanism or cached response.
- **Connection Refused** → Verify endpoint, firewall, and service availability.
- **SSL Handshake Failure** → Verify certificates and truststore.

Kafka Advanced

- **Broker Disk Full** → Increase storage and clean old topics.
- **ISR Shrinking** → Check broker/network health.
- **Under Replicated Partitions** → Investigate broker failures.
- **Offset Commit Issues** → Verify consumer commit strategy.

Kubernetes Advanced

- **Readiness Probe Failure** → Application not ready to serve traffic.
- **Liveness Probe Failure** → Application unhealthy and restarted.
- **HPA Not Scaling** → Verify metrics server and thresholds.
- **ImagePullBackOff** → Check image name, tag, and registry access.

AWS Advanced

- **Auto Scaling Not Triggered** → Verify CloudWatch metrics and policies.
- **High AWS Cost** → Analyze unused resources and overprovisioning.
- **S3 Latency Issues** → Optimize object access patterns.
- **IAM Permission Denied** → Verify role and policy configuration.

Logging / Monitoring

- **Disk Full Due To Logs** → Implement log rotation and retention.
- **Missing Correlation ID** → Add request tracing across services.
- **Difficult Root Cause Analysis** → Use distributed tracing.

Deployment Issues

- **Configuration Drift** → Keep configs version-controlled.
- **Blue-Green Failure** → Roll back traffic to previous environment.
- **Canary Failure** → Stop rollout and revert deployment.

S3 Issues

- **Access Denied on S3** → Verify IAM role and bucket policy.
- **File upload failing** → Check permissions and object size limits.
- **Objects deleted accidentally** → Enable versioning.
- **High S3 cost** → Implement lifecycle policies.
- **Slow file downloads** → Use CloudFront CDN.

EC2 Issues

- **EC2 CPU 100%** → Analyze application threads and scale instances.
- **EC2 memory exhausted** → Increase memory or fix application memory leak.
- **EC2 disk full** → Clean logs, temp files, and increase storage.
- **EC2 not reachable** → Verify Security Group, NACL, route table, and instance status.
- **EC2 restarted unexpectedly** → Check AWS events and system logs.
- **Application slow on EC2** → Monitor CPU, memory, disk, and network metrics.

Load Balancer Issues

- **ALB returning 502** → Backend application not responding correctly.
- **ALB returning 503** → No healthy targets available.
- **Targets unhealthy** → Verify health check endpoint and application status.
- **Traffic not reaching instances** → Check target group registration.

Auto Scaling Issues

- **Auto Scaling not triggering** → Verify CloudWatch alarms and scaling policies.
- **Instances not terminating** → Check scaling policy configuration.
- **Scaling too slowly** → Tune thresholds and cooldown periods.
- **Frequent scale-in/scale-out** → Adjust scaling metrics and cooldown settings.

RDS Issues

- **RDS CPU high** → Optimize queries and indexing.
- **RDS connections exhausted** → Increase pool size and fix connection leaks.
- **RDS storage full** → Increase storage allocation.
- **Slow RDS performance** → Analyze slow query logs.
- **RDS failover occurred** → Check Multi-AZ and database events.
- **Read replica lag** → Investigate heavy write traffic.

Java Scripts

JavaScript — A dynamic, interpreted, single-threaded scripting language primarily used to add interactivity to web pages.

ECMAScript (ES) — The official standard specification that defines the core features and syntax of the JavaScript language.

Variable — A named container that stores a value which can be referenced and manipulated throughout the program.

var — Function-scoped or globally-scoped variable declaration that is hoisted to the top of its scope.

let — Block-scoped variable declaration introduced in ES6 that allows reassignment but not re-declaration in the same scope.

const — Block-scoped variable declaration that cannot be reassigned.

Hoisting — JavaScript's behavior of moving variable and function declarations to the top of their scope before execution.

Scope — The region of code where a variable or function is accessible and valid.

Global Scope — Variables accessible throughout application.

Function Scope — Variables accessible only inside function.

Block Scope — Variables accessible only inside {} block.

Closure — Function remembering variables from outer scope even after execution.

Callback Function — A function passed as an argument to another function to be invoked at a later point in time.

Higher Order Function — A function that takes one or more functions as arguments or returns a function as its result.

Arrow Function — A concise function syntax introduced in ES6 using => that does not bind its own this.

Anonymous Function — A function without a name, typically assigned to a variable or passed as an argument.

IIFE — Immediately Invoked Function Expression; a function that is defined and called immediately upon creation.

Promise — An object representing the eventual completion or failure of an asynchronous operation, with .then() and .catch().

Async/Await — Cleaner syntax for handling asynchronous operations.

Event Loop — JavaScript's runtime mechanism that picks tasks from the callback queue and executes them on the call stack.

Call Stack — A LIFO data structure that tracks the currently executing function and the sequence of function calls.

Microtask Queue — A higher-priority queue (than the callback queue) for resolved Promises and mutation observer callbacks.

Callback Queue — A queue (also called macro-task queue) holding callbacks from setTimeout, setInterval, and I/O events.

Synchronous — Code that executes sequentially, line by line, blocking the thread until each operation completes.

Asynchronous — Code that initiates an operation and continues execution without waiting for the operation to finish.

setTimeout() — Executes function after specified delay.

setInterval() — Repeatedly executes a function at a specified interval in milliseconds until cleared.

clearTimeout() — Cancels a scheduled setTimeout before the callback has had a chance to execute.

clearInterval() — Stops a setInterval from executing any further callbacks.

DOM — Document Object Model; a tree-structured, in-memory representation of the HTML document that JavaScript can manipulate.

BOM — Browser Object Model; provides JavaScript access to browser features like window, navigator, location, and history.

Event Bubbling — An event triggered on a child element propagates upward through its ancestors in the DOM tree.

Event Capturing — Event propagates from parent to child.

Event Delegation — Handling child events using parent listener.

addEventListener() — Attaches event handler to element.

this Keyword — A special keyword whose value is determined by how and where a function is called; refers to the current context.

call() — Invokes a function with a specified this value and individual arguments passed comma-separated.

apply() — Invokes a function with a specified this value and arguments passed as an array.

bind() — Returns a new function permanently bound to a specified this value and optional preset arguments.

Prototype — An internal object that every JavaScript object inherits from, enabling shared properties and method inheritance.

Prototype Chain — The chain of prototype links traversed when looking up a property not found on the immediate object.

Class — ES6 syntactic sugar over prototype-based inheritance for defining objects with constructor and methods.

Object — A collection of key-value pairs (properties and methods) that can represent real-world entities.

Array — Ordered collection of values.

Map — Key-value collection allowing any datatype as key.

Set — Collection storing unique values only.

WeakMap — Map holding weak object references.

WeakSet — Set holding weak object references.

Destructuring — A syntax for unpacking values from arrays or properties from objects into distinct variables.

Spread Operator (...) — Expands an iterable (array, object) into individual elements in function calls or literals.

Rest Operator (...) — Collects the remaining arguments in a function parameter or remaining elements in destructuring into an array.

Template Literals — String interpolation using backticks.

Optional Chaining (?.) — Safely accesses deeply nested object properties, returning undefined instead of throwing if a reference is null.

Nullish Coalescing (??) — Returns right value only if left is null/undefined.

Truthy Values — Values that evaluate to true in a boolean context (everything except the six falsy values).

Falsy Values — The six values that evaluate to false: false, 0, "", null, undefined, and NaN.

Type Coercion — Automatic datatype conversion by JavaScript.

== Operator — Loose equality operator that compares values after performing type coercion if types differ.

=== Operator — Strict equality operator that compares both value and type without any coercion.

NaN — Represents invalid numeric value.

Undefined — Variable declared but not assigned.

Null — Intentional absence of value.

Deep Copy — Creates completely independent object copy.

Shallow Copy — Copies an object's top-level properties; nested objects still share the same reference.

Local Storage — Browser storage persisting even after browser close.

Session Storage — Browser storage cleared after session ends.

Cookie — Small text data stored in the browser and sent automatically with every HTTP request to the server.

Debouncing — Delays function execution until user stops triggering event.

Throttling — Limits function execution frequency.

Memoization — Optimization technique that caches function return values for previously seen inputs to avoid recomputation.

Currying — Converting function with multiple arguments into nested single-argument functions.

Pure Function — A function that always returns the same output for the same input and produces no side effects.

Side Effect — Operation affecting external state outside function.

Module — Reusable JavaScript file with isolated scope.

Export — Makes variables/functions accessible outside module.

Import — Uses exported members from another module.

JSON — Lightweight data interchange format.

AJAX — Asynchronous browser-server communication without page reload.

Fetch API — Modern browser API for making HTTP requests that returns Promises, replacing XMLHttpRequest.

CORS — Cross-Origin Resource Sharing; a browser security mechanism that controls HTTP requests to a different origin.

Strict Mode — Enables stricter JavaScript parsing and error handling.

Garbage Collection — Automatic memory cleanup for unused objects.

React

React — JavaScript library used for building component-based user interfaces.

Component — Reusable independent UI building block in React.

Functional Component — React component written using JavaScript function.

Class Component — Older React component syntax using ES6 classes.

JSX — HTML-like syntax used inside JavaScript in React.

Props — Read-only data passed from parent to child component.

State — Mutable data local to a component that, when updated, triggers a re-render of the component.

useState — Hook used to manage state in functional components.

useEffect — Hook used for side effects like API calls and subscriptions.

Dependency Array — The second argument to useEffect that controls when the effect runs; empty array = run once, filled = run on change.

useRef — Hook that returns a mutable ref object persisting across renders without causing re-renders when changed.

useMemo — Hook that memoizes an expensive computed value, recomputing it only when specified dependencies change.

useCallback — Hook that memoizes a function reference, preventing it from being recreated on every render.

useContext — Accesses shared global data without prop drilling.

Context API — React mechanism for sharing state globally across the component tree without passing props at every level.

Prop Drilling — Passing props through multiple nested components unnecessarily.

Virtual DOM — Lightweight copy of real DOM used for efficient updates.

Reconciliation — React process comparing Virtual DOM changes efficiently.

Key Prop — A unique identifier on list items that helps React efficiently identify which items changed, were added, or removed.

Conditional Rendering — Rendering UI based on conditions.

Controlled Component — Form element controlled by React state.

Uncontrolled Component — Form element managed using DOM refs directly.

Lifting State Up — Moving shared state to common parent component.

React Fragment — Groups multiple elements without extra DOM node.

React Router — Library used for client-side routing in React apps.

Route — Defines component mapping for specific URL path.

BrowserRouter — Enables browser history-based routing.

Link — Navigates between routes without page refresh.

Navigate — React Router component used for programmatic or declarative redirection to another route.

useNavigate — Hook for navigation programmatically.

Redux — A predictable state management library for JavaScript apps that maintains a single global store.

Store — Centralized state container in Redux.

Action — Object describing state change intention.

Reducer — Pure function updating Redux state.

Dispatch — Sends actions to Redux store.

Middleware — Functions that sit between dispatching an action and the reducer, used for logging, async calls, etc.

Redux Thunk — Middleware handling asynchronous Redux actions.

Redux Toolkit — Simplified modern Redux development approach.

React Query — Library for managing server-state (fetching, caching, syncing, updating) in React applications.

Custom Hook — A JavaScript function whose name starts with "use" that encapsulates and reuses stateful logic across components.

Higher Order Component (HOC) — A function that takes a component and returns a new enhanced component with additional behavior.

Render Props — Sharing logic using function props.

Pure Component — Component avoiding unnecessary re-renders using shallow comparison.

React.memo — Prevents re-rendering when props do not change.

Lazy Loading — Loading components only when needed.

Suspense — Displays fallback UI while lazy-loaded component loads.

Code Splitting — Breaking application bundle into smaller chunks.

Error Boundary — Catches JavaScript errors in component tree.

Synthetic Event — Cross-browser wrapper around native browser events.

Refs — Direct access to DOM elements/components.

Forward Ref — Passes refs from parent to child component.

Portal — Renders component outside normal parent DOM hierarchy.

StrictMode — Tool highlighting potential React problems during development.

Hydration — Attaching React functionality to server-rendered HTML.

SSR — Server Side Rendering generating HTML on server.

CSR — Client Side Rendering rendering UI in browser.

Next.js — React framework supporting SSR and full-stack features.

Vite — Fast frontend build tool commonly used with React.

Webpack — Module bundler for React applications.

Babel — JavaScript compiler converting modern JS into browser-compatible code.

npm — Package manager for installing JavaScript libraries.

yarn — Alternative package manager for JavaScript projects.

State Management — Managing application data flow and updates.

Immutability — Avoiding direct modification of existing state objects.

One-Way Data Binding — Data flows only from parent to child.

Hooks — Functions enabling state and lifecycle features in functional components.

useLayoutEffect — Runs synchronously after DOM updates before browser paint.

useReducer — Hook managing complex state logic using reducers.

useImperativeHandle — Customizes ref values exposed to parent.

React Fiber — React's internal rendering engine improving UI responsiveness.

Concurrent Rendering — React rendering feature improving UI performance and responsiveness.

Batching — Grouping multiple state updates into single re-render.

Hydration Error — Mismatch between server-rendered and client-rendered HTML.

Memoization — Caching results to improve performance.

Debouncing — Delaying rapid event execution like search input.

Throttling — Limiting function execution frequency during events.

Profiler — React tool for measuring rendering performance.

HTML

HTML — Standard markup language used to structure web pages.

Element — Building block of HTML defined using tags.

Tag — Syntax enclosed in angle brackets defining HTML elements.

Attribute — Additional information provided inside HTML tags.

DOCTYPE — Declares HTML document type and version.

Semantic HTML — Meaningful HTML elements improving readability and SEO.

div — Generic block-level container element.

span — Generic inline container element.

Heading Tags — <h1> to </h1><h6> representing headings hierarchy.

Paragraph Tag — <p> used for paragraphs/text blocks.

Anchor Tag — <a> used for hyperlinks/navigation.

Image Tag — used for displaying images.
Form — HTML element used for collecting user input.
Input Tag — <input> used for user data entry.
Label Tag — Associates label with form input.
Button Tag — Creates clickable button element.
Table — Displays tabular data using rows and columns.
List — Displays ordered or unordered collection of items.
Iframe — Embeds another webpage/document inside current page.
Meta Tag — Provides metadata about webpage.
Viewport Meta Tag — Controls responsive scaling on devices.
SEO — Optimizing webpage visibility in search engines.
Accessibility — Making websites usable for all users including disabilities.
ARIA — Accessibility attributes improving screen-reader support.
Inline Element — Occupies only required content width.
Block Element — Occupies full available width.
Inline-Block Element — Behaves inline while supporting block properties.
Local Storage — Browser storage persisting after browser close.
Session Storage — Browser storage cleared after session ends.
Cookie — Small browser-stored data sent with requests.
Canvas — HTML element used for drawing graphics dynamically.
SVG — XML-based scalable vector graphics format.
Audio Tag — Embeds audio content in webpage.
Video Tag — Embeds video content in webpage.
Responsive Design — UI adapting to different screen sizes.
DOM — Document Object Model representing webpage structure.

CSS

CSS — Stylesheet language used to design HTML webpages.
Selector — Targets HTML elements for styling.
Class Selector — Targets elements using class attribute.
ID Selector — Targets unique element using ID.
Universal Selector — Targets all HTML elements.
Pseudo Class — Styles element based on state like :hover.
Pseudo Element — Styles specific part of element like ::before.
Box Model — Structure consisting of margin, border, padding, and content.
Margin — Space outside element border.
Padding — Space between content and border.
Border — Outline surrounding element.
Content — Actual text/image inside element.
Display Property — Defines element rendering behavior.
display:block — Element occupies full width.
display:inline — Element occupies only content width.
display:flex — Enables Flexbox layout system.
display:grid — Enables CSS Grid layout system.
Flexbox — One-dimensional layout system for alignment/distribution.
CSS Grid — Two-dimensional layout system using rows and columns.
Position Property — Defines element positioning behavior.
Static Position — Default normal document positioning.
Relative Position — Positioned relative to normal location.
Absolute Position — Positioned relative to nearest positioned ancestor.
Fixed Position — Positioned relative to browser viewport.
Sticky Position — Switches between relative and fixed based on scrolling.
z-index — Controls stacking order of elements.
Media Query — Applies styles based on device/screen conditions.
Responsive Web Design — Adapting UI for multiple devices/screens.
Viewport — Visible webpage area in browser/device.
Specificity — Rule deciding which CSS style gets applied.
Inheritance — Child elements inheriting parent styles automatically.
!important — Forces CSS rule priority override.
Transition — Smooth property change effect over time.
Animation — Complex visual movement/effects using keyframes.

Transform — Rotates, scales, skews, or moves elements visually.
Opacity — Controls element transparency.
Overflow — Defines handling of extra overflowing content.
Visibility:hidden — Hides element while keeping layout space.
display:none — Removes element completely from layout.
Float — Positions elements left or right.
Clear — Prevents floating element overlap.
CSS Variables — Reusable custom CSS property values.
Rem Unit — Relative unit based on root font size.
Em Unit — Relative unit based on parent font size.
Viewport Units — Relative sizing based on viewport dimensions (vw, vh).
Bootstrap — Popular CSS framework for responsive UI development.
Tailwind CSS — Utility-first CSS framework for rapid styling.
Preprocessor — CSS extension language like SCSS/SASS.
SCSS/SASS — CSS preprocessor adding variables, nesting, and functions.
BEM Methodology — CSS naming convention improving maintainability.
CSS Reset — Removes default browser styling inconsistencies.
Normalize CSS — Standardizes browser default styles consistently.
Shadow DOM — Encapsulated DOM/CSS used in web components.

React React Production Issues & Fixes (One-Liners)

Performance Issues

- **Page loading slow** → Use code splitting and lazy loading.
- **Large bundle size** → Enable tree shaking and bundle optimization.
- **Frequent re-renders** → Use `React.memo`, `useMemo`, and `useCallback`.
- **UI freezing** → Move heavy calculations outside render cycle.
- **Slow search/filtering** → Use debouncing and memoization.
- **Large list rendering slow** → Use virtualization (`react-window`, `react-virtualized`).
- **Memory usage increasing** → Clean up subscriptions and event listeners.

API Issues

- **API calls firing multiple times** → Check `useEffect` dependencies.
- **Duplicate API requests** → Prevent multiple clicks and request retries.
- **API response slow** → Add loading states and caching.
- **API timeout** → Configure timeout and retry mechanism.
- **429 Too Many Requests** → Implement throttling and rate limiting.
- **Network failure** → Show fallback UI and retry option.

State Management Issues

- **State not updating** → Avoid direct state mutation.
- **UI not reflecting changes** → Verify immutable updates.
- **Redux state lost after refresh** → Persist state using `localStorage/sessionStorage`.
- **Prop drilling issue** → Use Context API or Redux.
- **Global state becoming large** → Split state into feature modules.

useEffect Issues

- **Infinite API calls** → Fix dependency array.
- **Infinite re-render loop** → Avoid state updates inside uncontrolled effects.
- **Memory leak in useEffect** → Return cleanup function.
- **Stale data issue** → Include correct dependencies.

Routing Issues

- **Page refresh causing 404** → Configure server-side route fallback.
- **Navigation not working** → Verify route configuration.
- **Protected route bypassed** → Add authentication guards.
- **Route loading slow** → Lazy load routes.

Authentication Issues

- **User logged out after refresh** → Store token securely.
- **JWT expired** → Implement token refresh mechanism.
- **Unauthorized API calls (401)** → Verify token sent in headers.
- **Role-based access not working** → Verify authorization logic.

Browser Issues

- **Application works in Chrome but not Firefox** → Check browser compatibility.
- **UI breaking on mobile** → Use responsive CSS and media queries.
- **LocalStorage not working** → Verify browser privacy/security restrictions.
- **CORS issue** → Configure backend CORS settings properly.

React Build & Deployment Issues

- **Build failing** → Check compilation and dependency errors.
- **Environment variables not loading** → Use correct `.env` naming conventions.
- **Application working locally but not production** → Verify production configurations.
- **Static assets not loading** → Verify asset paths and CDN configuration.
- **Source maps exposed** → Disable source maps in production builds.

Memory Leak Issues

- **Event listener leak** → Remove listeners during component unmount.
- **Timer leak** → Clear `setTimeout` and `setInterval`.
- **WebSocket leak** → Close connections during cleanup.
- **Subscription leak** → Unsubscribe in cleanup function.

UI Issues

- **Button clicked multiple times** → Disable button during API call.
- **Form submitted twice** → Prevent duplicate submission.
- **Loader never stops** → Handle success and failure cases properly.
- **Blank page in production** → Check browser console and application logs.
- **Unexpected component re-render** → Use memoization techniques.

Security Issues

- **XSS vulnerability** → Sanitize user input and avoid unsafe HTML rendering.
- **Sensitive data exposed in frontend** → Never store secrets in React app.
- **Token exposed in URL** → Store tokens securely in headers/cookies.
- **API endpoint exposed** → Protect APIs with authentication and authorization.

CDN / Caching Issues

- **Users seeing old UI after deployment** → Implement cache busting/versioning.
- **Static files cached incorrectly** → Configure proper cache headers.
- **Deployment successful but UI unchanged** → Clear CDN/browser cache.

Monitoring & Logging

- **Frontend errors not tracked** → Integrate monitoring tools like Sentry.
- **Hard to debug production issue** → Add structured logging and error tracking.
- **Performance degradation** → Analyze browser performance metrics.

Top 15 Most Asked React Production Issues

1. Infinite API calls → Wrong `useEffect` dependency.
2. Excessive re-renders → Use memoization.
3. Slow page load → Lazy loading + code splitting.
4. Large bundle size → Tree shaking and optimization.
5. Memory leak → Cleanup effects/listeners.
6. State not updating → Avoid mutation.
7. Duplicate API calls → Debounce and prevent multiple clicks.
8. JWT expiration → Refresh token flow.
9. Blank production screen → Check console/build errors.
10. Page refresh 404 → Configure server routing.
11. Large list performance issue → Virtualization.
12. CORS issue → Backend CORS configuration.
13. Users seeing old UI → Cache busting.
14. Build works locally but fails in production → Environment/config mismatch.
15. UI freezing → Move heavy computation off render path.

Interview Shortcut

For any React production issue, answer in this order:

Why? → Rendering, state, API, cache, routing, or browser issue.

Investigate? → Browser console, Network tab, React DevTools, logs.

Fix? → Memoization, cleanup, caching, lazy loading, proper state management, or configuration correction.

Browser Performance

- **Page becomes slow after long usage** → Check memory leaks and excessive DOM nodes.
- **High browser memory usage** → Remove unused references and cleanup listeners.
- **Browser tab crashes** → Optimize large datasets and memory consumption.
- **High CPU usage in browser** → Reduce unnecessary renders and expensive calculations.

React Rendering Issues

- **Component re-rendering too often** → Use `React.memo`, `useMemo`, `useCallback`.
- **Parent re-render causing child re-renders** → Memoize child components.
- **Context API causing full app re-render** → Split contexts by feature.
- **Large Redux store causing slowness** → Normalize and modularize state.

WebSocket / Real-Time Issues

- **WebSocket disconnects frequently** → Implement reconnect strategy.
- **Real-time updates delayed** → Check network latency and message queue.
- **Duplicate real-time events** → Implement deduplication logic.

File Upload Issues

- **Large file upload failing** → Use chunked uploads.
- **Upload taking too long** → Compress files before upload.

- **Multiple uploads freezing UI** → Upload asynchronously.

Accessibility (A11Y)

- **Screen reader not reading content** → Add proper ARIA attributes.
- **Keyboard navigation broken** → Support tab and keyboard events.
- **Accessibility audit failures** → Follow WCAG guidelines.

Responsive UI Issues

- **Layout breaks on mobile** → Use responsive CSS and media queries.
- **Text overflow issue** → Apply proper wrapping and overflow handling.
- **Different behavior across devices** → Test on multiple screen sizes.

Micro Frontend Issues

- **Module not loading** → Verify remote entry configuration.
- **Version conflict between apps** → Share common dependencies.
- **Cross-app communication issue** → Use event bus/shared state.

Security Issues

- **Cross-Site Scripting (XSS)** → Sanitize user input.
- **CSRF attack risk** → Use CSRF tokens.
- **Sensitive data exposed in browser** → Never store secrets in frontend.

Monitoring & Observability

- **Cannot reproduce production issue** → Add client-side logging.
- **No visibility into user failures** → Use Sentry/New Relic/Datadog.
- **Frontend performance issue** → Track Core Web Vitals.

Lambda Issues

- **Lambda timeout** → Optimize code or increase timeout setting.
- **Cold start delay** → Use provisioned concurrency.
- **Lambda concurrency limit reached** → Increase concurrency quota.
- **Lambda memory exceeded** → Increase memory allocation.
- **Lambda unable to access resources** → Verify IAM permissions.

SQS Issues

- **Queue backlog increasing** → Scale consumers.
- **Messages stuck in queue** → Check consumer health.
- **Duplicate messages** → Implement idempotent processing.
- **Messages moving to DLQ** → Analyze consumer failures.
- **Message processing slow** → Increase consumer instances.

SNS Issues

- **Notifications not delivered** → Verify subscriptions and permissions.
- **Email/SMS not received** → Check subscription confirmation status.
- **SNS publishing failure** → Verify IAM policies.

MSK / Kafka Issues

- **Broker unavailable** → Check broker health and networking.
- **Under-replicated partitions** → Investigate broker failures.
- **High consumer lag** → Scale consumers.
- **Producer timeout** → Verify broker connectivity.
- **Disk utilization high** → Expand broker storage.

IAM Issues

- **Access Denied errors** → Verify IAM policies and roles.
- **Role assumption failure** → Check trust relationship configuration.
- **Unexpected permission issues** → Validate policy inheritance and boundaries.

CloudWatch Issues

- **Metrics not visible** → Verify CloudWatch agent configuration.
- **Alerts not triggering** → Check thresholds and evaluation periods.
- **Missing logs** → Verify log group and IAM permissions.
- **High CloudWatch cost** → Optimize log retention.

VPC / Networking Issues

- **Service cannot reach internet** → Verify NAT Gateway and route tables.
- **Instances cannot communicate** → Check Security Groups and NACLs.
- **DNS resolution failure** → Verify Route 53 and VPC DNS settings.
- **Network latency high** → Check routing and availability zones.

Route 53 Issues

- **Domain not resolving** → Verify DNS records and nameservers.
- **Traffic routing incorrect** → Review routing policies.
- **DNS propagation delay** → Check TTL configuration.

Secrets Manager / Parameter Store

- **Application cannot read secrets** → Verify IAM permissions.
- **Secret rotation failure** → Check rotation Lambda execution.
- **Configuration not updating** → Refresh cached values.